

LAB 1 — Python Setup & Execution (with VS Code)

Objective

By the end of this lab, you will be able to:

- Install Python
- Verify installation
- Run Python interactively
- Execute a Python script
- Use Visual Studio Code for Python development



Python Development Environment Setup

Setup. Configure. Code. Run.



Visual Studio Code Setup
Extensions
Python
IntelliSense (Pylance),
Linting, Debugging
Installed
Python ✓
Pylance ✓
Python Debugger ✓
Jupyter ✓

main.py
1 # main.py
2 name = "Beginner"
3 print("Hello " + name)
4 print("Running Python Script")
5
6
TERMINAL
PS C:\Users\Beginner\project> python main.py
Running Python Script
PS C:\Users\Beginner\project> |

Run in Terminal
C:\Users\Beginner\project>
python main.py
Running Python Script ✓

Python Interactive Mode
C:\Users\Beginner> python
Python 3.12.3 (tags/v3.12.3)
Type "help", "copyright", "credits"
or "license" for more information.
>>> print("Hello Python")
Hello Python
>>> |

Project File Structure
my_project
├── main.py
└── README.md

main.py Write Code → Python Interpreter → Execute → Output

What You Need
Python Installed ✓
VS Code Installed ✓
Python Extension ✓
Ready to Code! 🚀 ✓



Tip: Use a virtual environment for your projects

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

(Windows)

Step-by-Step with Code

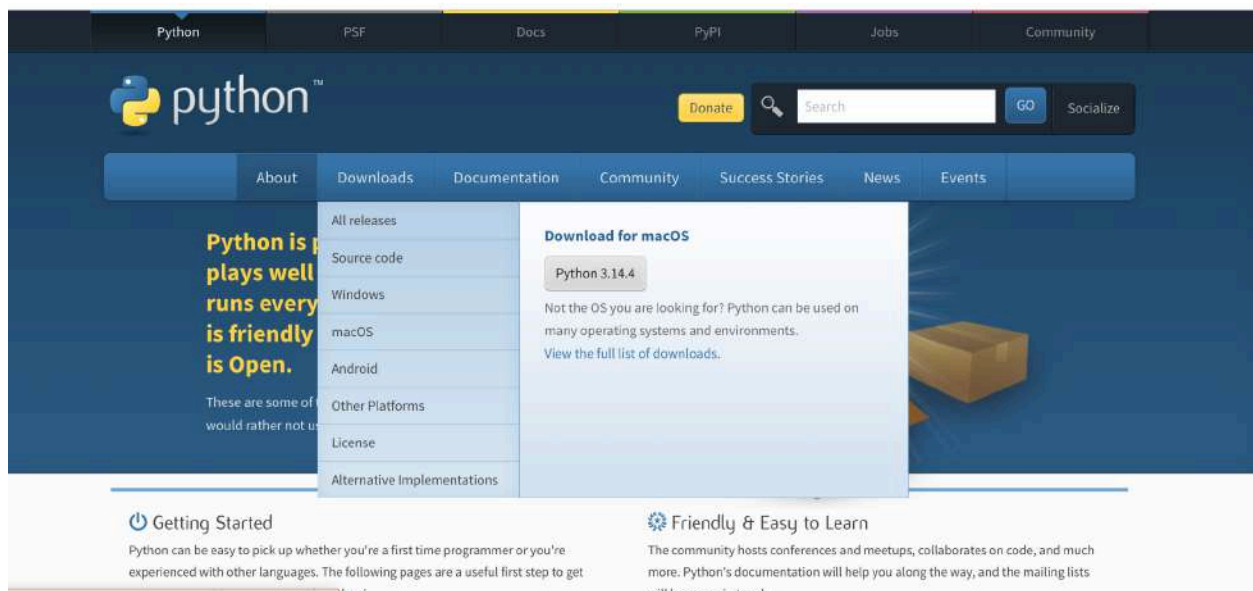
Step 1 — Install Python

Download from:

👉 <https://www.python.org>

Install with:

- ☒ “Add Python to PATH” checked



Step 2 — Verify Installation

```
python --version
```

OR

```
python3 --version
```

Expected Output:

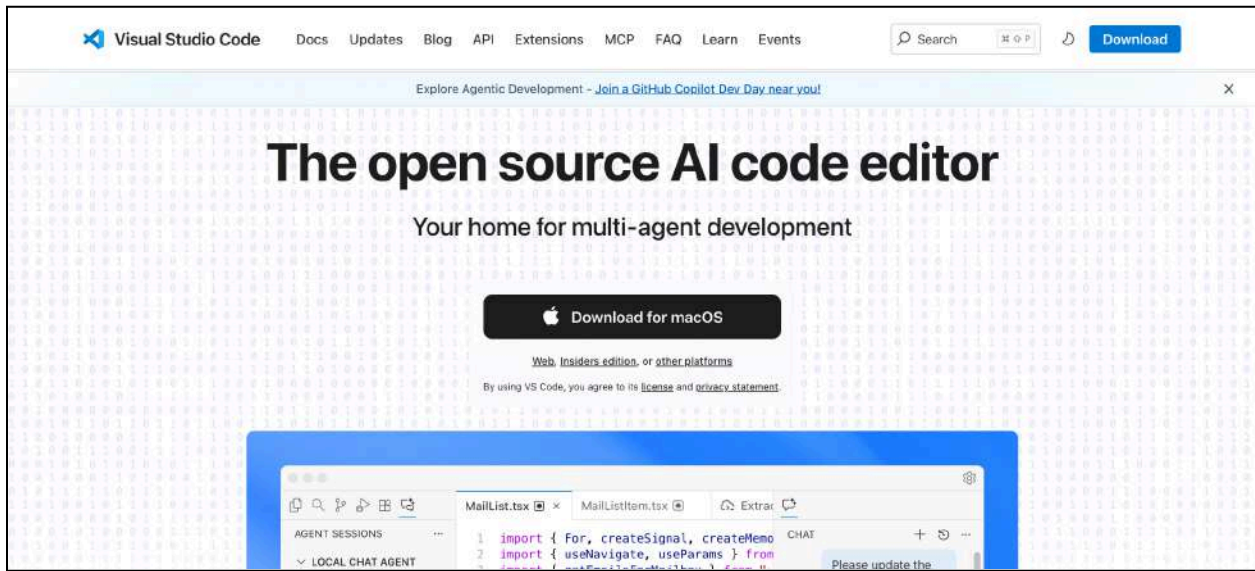
```
Python 3.x.x
```

Step 3 — Install VS Code

Download:

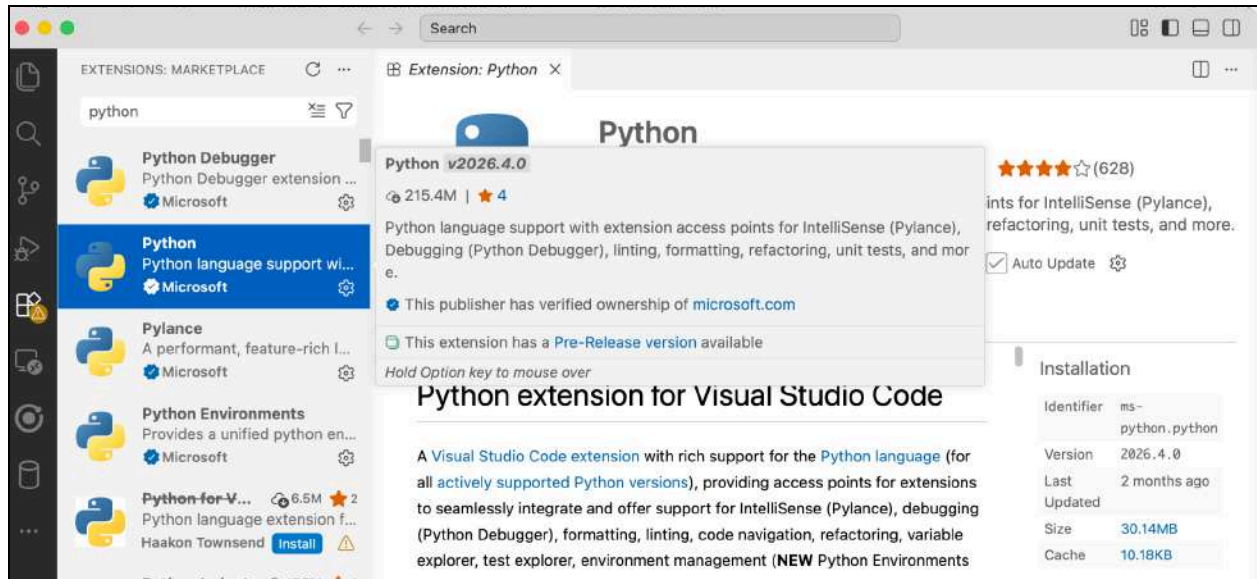
👉 <https://code.visualstudio.com>

Install normally (default settings are fine)



Step 4 — Install Python Extension in VS Code

1. Open **Visual Studio Code**
2. Go to **Extensions (left sidebar)**
3. Search: **Python**
4. Install extension by **Microsoft**



Step 5 — Run Python Interpreter

```
python
```

Then type:

```
print("Hello Python")
```

Output:

```
Hello Python
```

Step 6 — Create Script File

Create file:

```
main.py
```

Add:

```
print("Running Python Script")
```

Run:

```
python main.py
```

Output:

```
Running Python Script
```

Step 7 — Multiple Lines

```
name = "Student"  
print("Hello", name)
```

Optional Exercise

Create a script that:

- prints your name
- prints your favorite programming language

Answer

```
name = "Wan"  
language = "Python"  
  
print("Name:", name)  
print("Favorite Language:", language)
```

Key Takeaways

- Python is easy to install and run
- Interpreter allows quick testing
- Script mode is used for real programs
- Python executes code line-by-line

- Python 3 is the standard
- Foundation for all upcoming labs

LAB 2 — Variables & Input

Objective

By the end of this lab, you will be able to:

- Create and use variables
- Accept user input
- Display formatted output
- Understand basic data flow (Input → Process → Output)



Python Variables & User Input

Store data in variables and display output



Let's write a simple program!

```
hello.py
1 # Get user input
2 name = input("Enter your name: ")
3 age = int(input("Enter your age: "))
4
5 # Create message
6 message = "Hello " + name
  age_message = "You are " + str(age) + " years old"
7
8 # Display output
9 print(message)
10 print(age_message)
```

What this program does

- Takes input from user
- Stores in variables
- Combines data
- Displays output

1 USER INPUT

User provides data

Enter your name:

Enter your age:

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
```

2 STORED IN VARIABLES

Data saved in variables

Variables

name
"Ali"

age
20

```
# Variables store data
name = "Ali"
age = 20
```

3 PROCESS

Combine data to create messages

"Hello " + name
("Ali")

 +

"You are " + str(age)
(20)
" years old"

Combined Messages
"Hello Ali"
"You are 20 years old"

```
# Combine data
message = "Hello " + name
age_message = "You are " + str(age) + " years old"
```

4 OUTPUT DISPLAY

Show results to the user

Output


```
# Display output
print(message)
print(age_message)
```

KEY CONCEPTS

- `input()` gets data from user
- Variables store the data
- We can combine strings and variables
- `print()` shows the result

VARIABLES IN MEMORY



DATA FLOW



OUTPUT EXAMPLE

```
Hello Ali
You are 20 years old
>_
```


Step-by-Step with Code

Step 1 — Create Variables

```
name = "Ali"  
age = 20  
  
print(name)  
print(age)
```

Output:

```
Ali  
20
```

Step 2 — Combine Variables

```
name = "Ali"  
age = 20  
  
print("Name:", name)  
print("Age:", age)
```

Step 3 — User Input

```
name = input("Enter your name: ")  
print("Hello", name)
```

Step 4 — Multiple Inputs

```
name = input("Enter your name: ")
```

```
age = input("Enter your age: ")

print("Name:", name)
print("Age:", age)
```

Step 5 — Simple Processing

```
name = input("Enter your name: ")
age = input("Enter your age: ")

print("Hello", name)
print("You are", age, "years old")
```

Optional Exercise

Create a program that:

- asks for name and favorite hobby
- prints a sentence describing the user

Answer

```
name = input("Enter your name: ")
hobby = input("Enter your hobby: ")

print(name, "likes", hobby)
```

Key Takeaways

- Variables store data
- Input allows user interaction
- Output displays results
- Python treats input as text by default
- Basic programs follow Input → Process → Output
- Foundation for building real applications

LAB 3 — String Processing

Objective

By the end of this lab, you will be able to:

- Work with strings
- Extract and manipulate text
- Perform basic string operations
- Build simple text-processing logic

Step-by-Step with Code

Step 1 — Create a String

```
text = "Hello Python"  
print(text)
```

Step 2 — String Length

```
text = "Hello Python"  
print(len(text))
```

Output:

```
12
```

Step 3 — Access Characters (Indexing)

```
text = "Python"  
  
print(text[0]) # P  
print(text[1]) # y
```

Step 4 — Extract Substring (Slicing)

```
text = "Hello Python"

print(text[0:5]) # Hello
print(text[6:12]) # Python
```

Step 5 — String Concatenation

```
first = "Hello"
second = "World"

result = first + " " + second
print(result)
```

Step 6 — Replace Text

```
text = "Hello Python"

new_text = text.replace("Python", "Developer")
print(new_text)
```

Step 7 — Search in String

```
text = "Hello Python"

print("Python" in text)
```

Output:

```
True
```

Optional Exercise

Create a program that:

- asks user for full name
- extracts first name
- prints greeting using first name

Answer

```
name = input("Enter your full name: ")  
  
first_name = name.split()[0]  
  
print("Hello", first_name)
```

Key Takeaways

- Strings store text data
- Indexing accesses individual characters
- Slicing extracts parts of text
- Strings can be combined and modified
- Built-in functions simplify processing
- Text processing is key for real applications

LAB 4 — Simple User Profile Application

Objective

By the end of this lab, you will be able to:

- Accept user input
- Process string data
- Format output using f-strings
- Build a simple real-world program



Python Application Workflow

Build a Simple Profile Program



Let's enter my details! 😊

Simple Profile Program

Name

Age

City

What this program does

Takes user input, processes the data, and displays a formatted message.

1 USER INPUT

User enters raw data.

Name

Age

City

2 DATA PROCESSING

Clean and transform the data.

TRIM
Remove extra spaces

CAPITALIZE
Make first letter uppercase

SPLIT
Split text into words (if needed)

```
# Get input from user
name = input("Name: ")
age = input("Age: ")
city = input("City: ")

# Process the data
name = name.strip().capitalize()
age = age.strip()
city = city.strip().capitalize()
city_words = city.split()

# Format output
message = f"Hello {name}! You are {age} years old. You live in {city}."
print(message)
```

3 FORMATTED OUTPUT

Display clean, user-friendly result.

Hello **Ali**!

You are **20** years old.

You live in **Kajang**.

FULL CODE OVERVIEW



```
name = input("Name: ").strip().capitalize()
age = input("Age: ").strip()
city = input("City: ").strip().capitalize()

message = f"Hello {name}! You are {age} years old. You live in {city}."
print(message)
```

- `input()` → Get user input
- `strip()` → Remove spaces
- `capitalize()` → Capitalize text
- `split()` → Split into words
- `f-string` → Format output

 **Tip:** Clean data → Better output. Always process user input before using it!



Step-by-Step with Code

Step 1 — Get User Input

```
name = input("Enter your name: ")
age = input("Enter your age: ")
city = input("Enter your city: ")
```

Step 2 — Clean Input

```
name = name.strip().title()
city = city.strip().title()
```

Step 3 — Process Data

```
first_name = name.split()[0]
```

Step 4 — Format Output

```
print(f"Hello {first_name}!")
print(f"You are {age} years old.")
print(f"You live in {city}.")
```

Step 5 — Full Program

```
name = input("Enter your name: ")
age = input("Enter your age: ")
city = input("Enter your city: ")

name = name.strip().title()
city = city.strip().title()

first_name = name.split()[0]
```



```
print(f"\n=== User Profile ===")
print(f"Name : {name}")
print(f"Age : {age}")
print(f"City : {city}")

print(f"\nHello {first_name}! Welcome!")
```

Expected Output (Example)

```
=== User Profile ===
Name : Ali Ahmad
Age : 20
City : Kajang

Hello Ali! Welcome!
```

Optional Exercise

Enhance the program to:

- add favorite hobby
- display full sentence summary

Answer

```
name = input("Enter your name: ")
age = input("Enter your age: ")
city = input("Enter your city: ")
hobby = input("Enter your hobby: ")

name = name.strip().title()
city = city.strip().title()
hobby = hobby.strip().lower()

first_name = name.split()[0]

print(f"\n=== User Profile ===")
print(f"Name : {name}")
print(f"Age : {age}")
```

```
print(f"City : {city}")  
print(f"Hobby: {hobby}")  
  
print(f"\n{first_name} is {age} years old and enjoys {hobby}.")
```

Key Takeaways

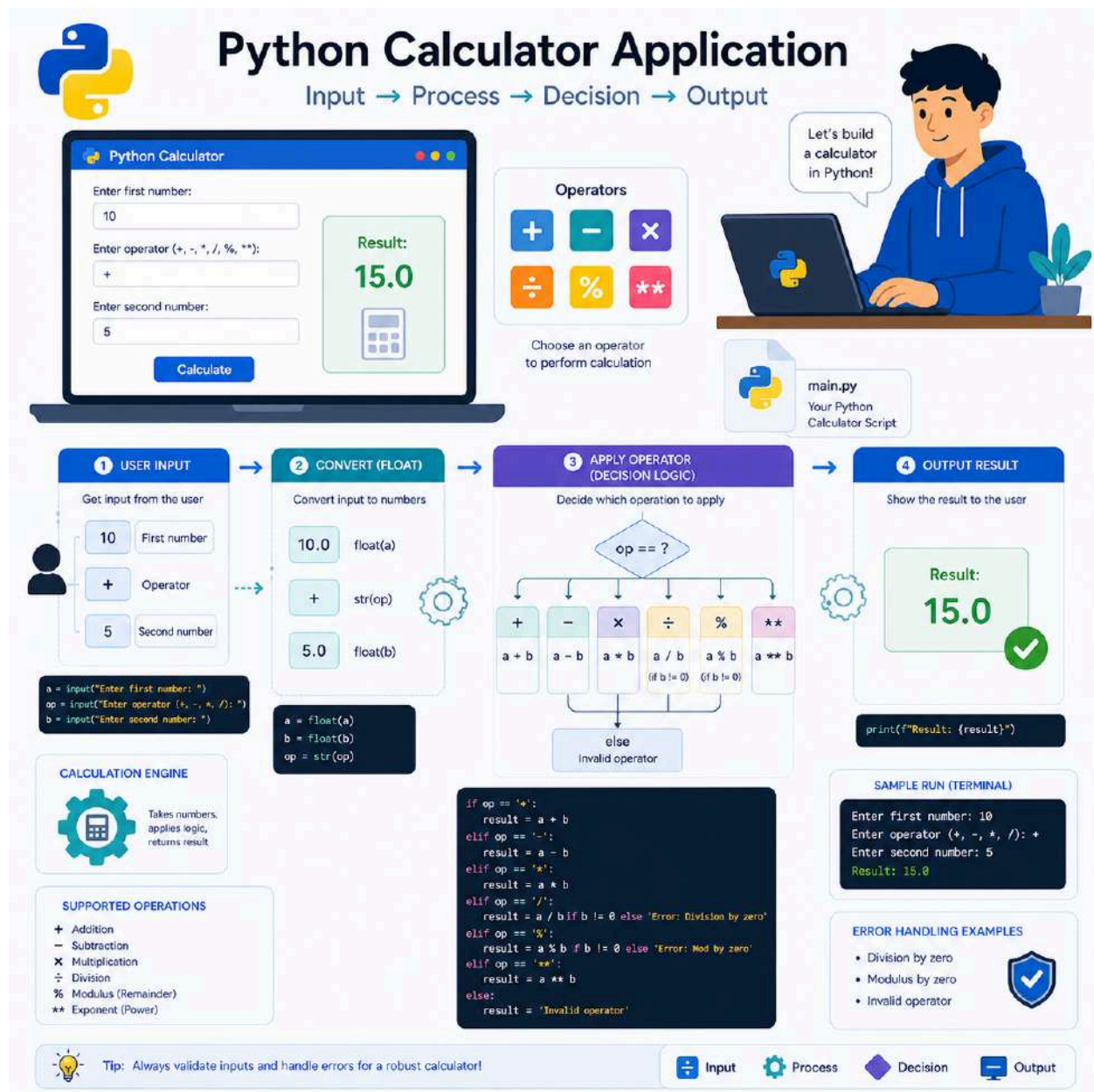
- Strings are core to user interaction
- f-strings make output clean and readable
- String methods simplify processing
- Input → Process → Output is key pattern
- Real applications combine multiple concepts
- You can already build simple usable programs

LAB 5 — Calculator Program

Objective

By the end of this lab, you will be able to:

- Accept numeric input from user
- Convert text input into numbers
- Perform arithmetic operations
- Display calculated results clearly



Step-by-Step with Code

Step 1 — Get Two Numbers

```
num1 = input("Enter first number: ")
num2 = input("Enter second number: ")

print(num1)
print(num2)
```

Step 2 — Convert Input to Float

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print(num1)
print(num2)
```

Step 3 — Add Two Numbers

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

result = num1 + num2

print("Result:", result)
```

Step 4 — Add More Operations

```
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)
print("Division:", num1 / num2)
```

Step 5 — Full Calculator Program

```
num1 = float(input("Enter first number: "))
operator = input("Enter operator (+, -, *, /): ")
num2 = float(input("Enter second number: "))

if operator == "+":
    result = num1 + num2
elif operator == "-":
    result = num1 - num2
elif operator == "*":
    result = num1 * num2
elif operator == "/":
    result = num1 / num2
else:
    result = "Invalid operator"

print("Result:", result)
```

Expected Output

```
Enter first number: 10
Enter operator (+, -, *, /): +
Enter second number: 5
Result: 15.0
```

Optional Exercise

Improve the calculator to:

- support modulus %
- support exponent **
- show error message for invalid operator

Answer

```
num1 = float(input("Enter first number: "))
operator = input("Enter operator (+, -, *, /, %, **): ")
num2 = float(input("Enter second number: "))
```

```
if operator == "+":
    result = num1 + num2
elif operator == "-":
    result = num1 - num2
elif operator == "*":
    result = num1 * num2
elif operator == "/":
    result = num1 / num2
elif operator == "%":
    result = num1 % num2
elif operator == "**":
    result = num1 ** num2
else:
    result = "Invalid operator"

print("Result:", result)
```

Key Takeaways

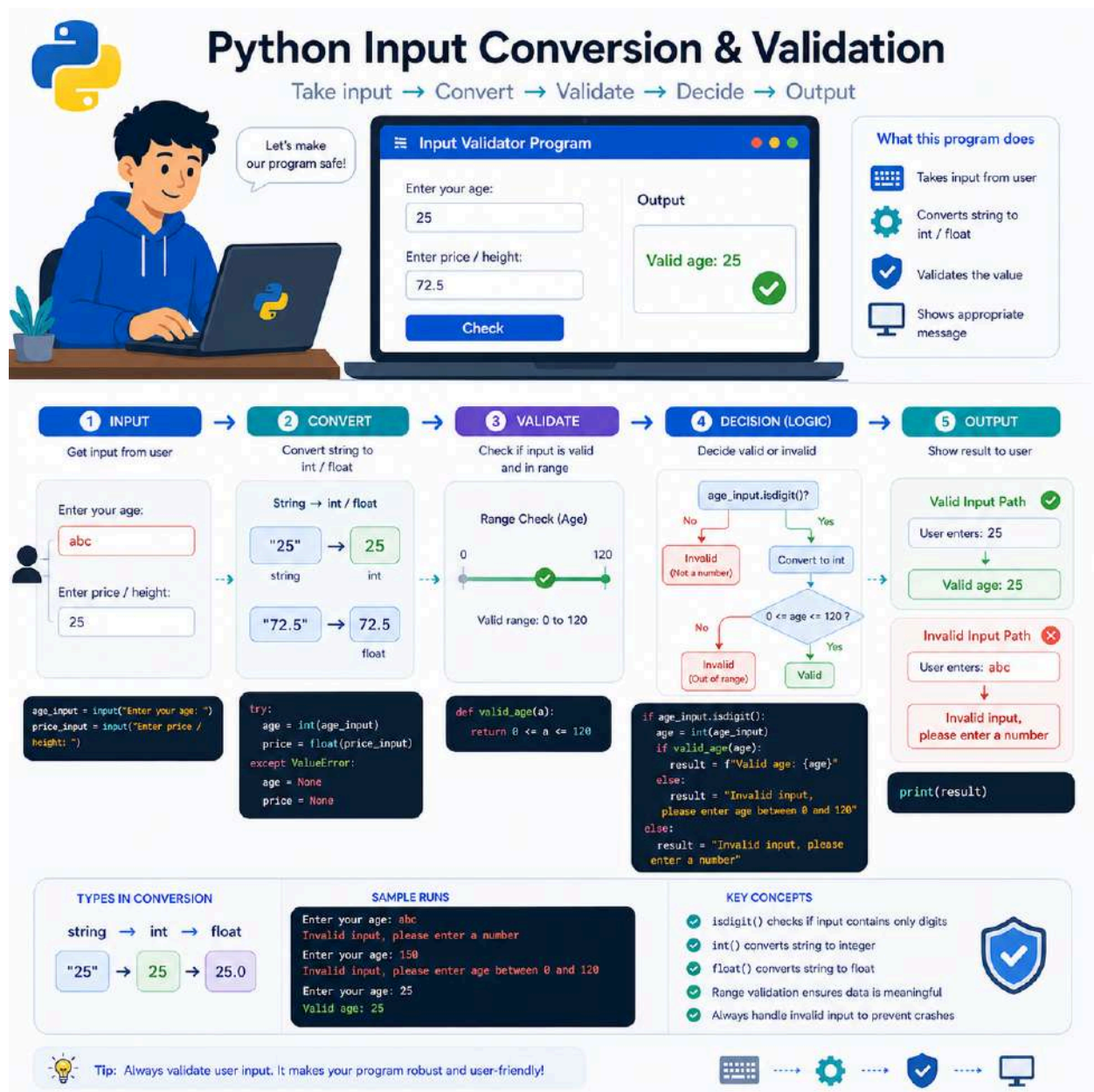
- `input()` always returns text
- Use `float()` for decimal calculation
- Operators perform arithmetic logic
- `if/elif/else` controls program flow
- Calculators combine input, conversion, and logic
- This is a foundation for decision-based apps

LAB 6 — Input Conversion & Validation

Objective

By the end of this lab, you will be able to:

- Convert user input from string to int/float
- Validate user input correctly
- Prevent program errors
- Build safe and reliable input handling



Step-by-Step with Code

Step 1 — Basic Input Problem

```
age = input("Enter your age: ")  
  
print(age + 10)
```

✗ Problem:

```
TypeError (cannot add string and number)
```

Step 2 — Convert String to Integer

```
age = int(input("Enter your age: "))  
  
print(age + 10)
```

✓ Output:

```
Enter your age: 20  
30
```

Step 3 — Convert to Float

```
price = float(input("Enter price: "))  
tax = price * 0.06  
  
print("Tax:", tax)
```

Step 4 — Handle Invalid Input (Basic Validation)

```
age = input("Enter your age: ")  
  
if age.isdigit():  
    age = int(age)
```



```
    print("Valid age:", age)
else:
    print("Invalid input")
```

Step 5 — Range Validation

```
age = int(input("Enter your age: "))

if age > 0 and age < 120:
    print("Valid age")
else:
    print("Invalid age range")
```

Step 6 — Full Input Validation Program

```
age_input = input("Enter your age: ")

if age_input.isdigit():
    age = int(age_input)

    if age > 0 and age < 120:
        print("Valid age:", age)
    else:
        print("Age out of valid range")
else:
    print("Invalid input, please enter a number")
```

Expected Output

```
Enter your age: 25
Valid age: 25
```

OR

```
Enter your age: abc
```

Invalid input, please enter a number

Optional Exercise

Enhance the program to:

- ask for height (float)
- validate both age and height
- display formatted summary

Answer

```
age_input = input("Enter your age: ")
height_input = input("Enter your height (m): ")

if age_input.isdigit():
    age = int(age_input)

    try:
        height = float(height_input)

        if age > 0 and age < 120 and height > 0:
            print(f"Age: {age}, Height: {height}m")
        else:
            print("Invalid range values")
    except:
        print("Invalid height input")
else:
    print("Invalid age input")
```

Key Takeaways

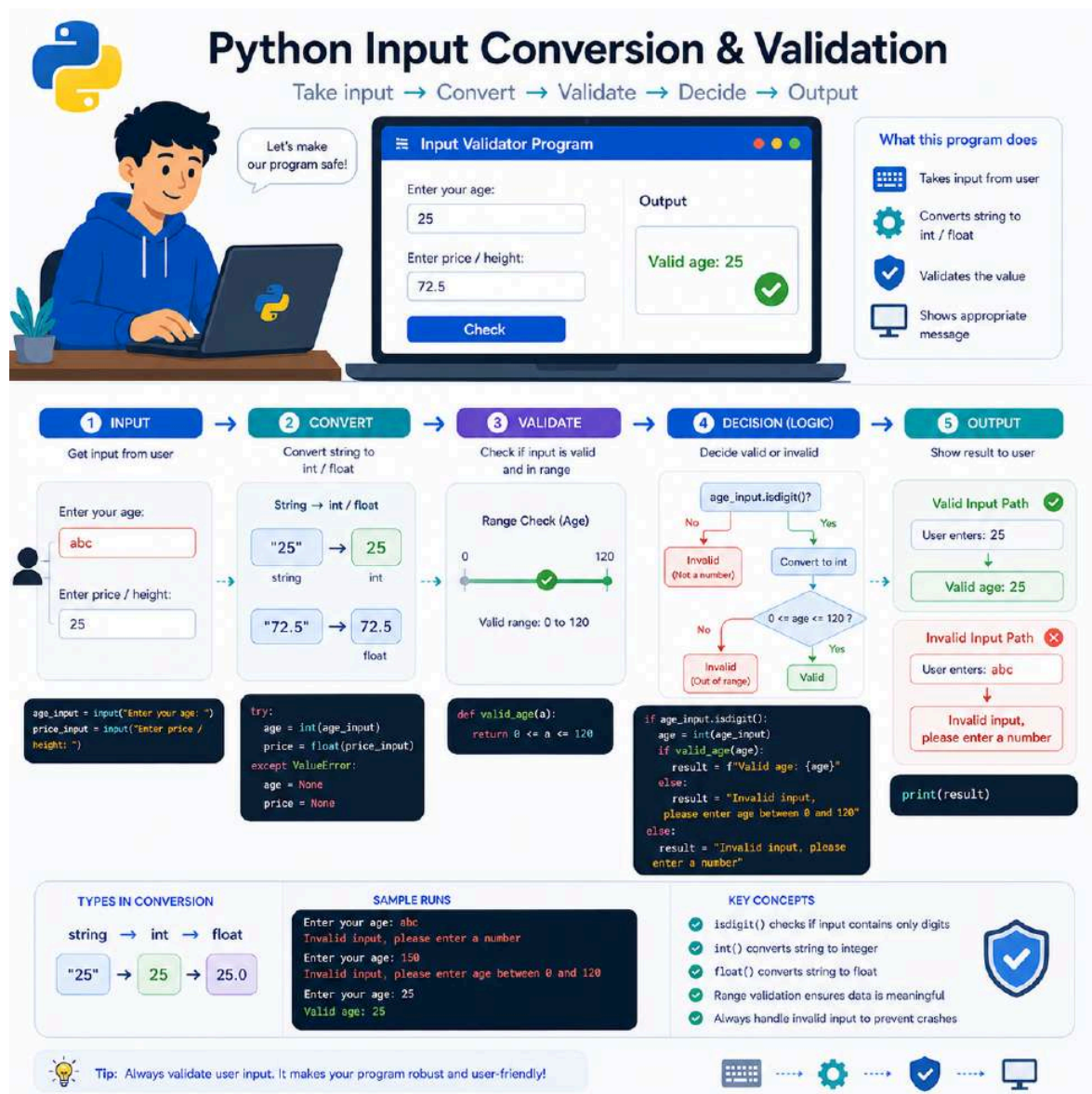
- `input()` always returns string
- Use `int()` and `float()` for conversion
- Validation prevents runtime errors
- `isdigit()` helps check numeric input
- Combine validation with conditions
- Safe input handling is critical in real applications

LAB 7 — Decision App

Objective

By the end of this lab, you will be able to:

- Accept score input from user
- Convert input into number
- Use `if`, `elif`, and `else`
- Apply grading rules
- Display clear decision output



Step-by-Step with Code

Step 1 — Get Score Input

```
score = input("Enter your score: ")  
  
print(score)
```

Step 2 — Convert Score to Integer

```
score = int(input("Enter your score: "))  
  
print("Score:", score)
```

Step 3 — Add Pass / Fail Logic

```
score = int(input("Enter your score: "))  
  
if score >= 50:  
    print("Pass")  
else:  
    print("Fail")
```

Step 4 — Add Grade Categories

```
score = int(input("Enter your score: "))  
  
if score >= 80:  
    print("Grade: A")  
elif score >= 70:  
    print("Grade: B")  
elif score >= 60:  
    print("Grade: C")  
elif score >= 50:  
    print("Grade: D")  
else:  
    print("Grade: F")
```

Step 5 — Add Score Validation

```
score = int(input("Enter your score: "))

if score < 0 or score > 100:
    print("Invalid score")
elif score >= 80:
    print("Grade: A")
elif score >= 70:
    print("Grade: B")
elif score >= 60:
    print("Grade: C")
elif score >= 50:
    print("Grade: D")
else:
    print("Grade: F")
```

Step 6 — Full Grading System

```
score_input = input("Enter your score: ")

if score_input.isdigit():
    score = int(score_input)

    if score < 0 or score > 100:
        print("Invalid score. Score must be between 0 and 100.")
    elif score >= 80:
        print("Result: Pass")
        print("Grade : A")
    elif score >= 70:
        print("Result: Pass")
        print("Grade : B")
    elif score >= 60:
        print("Result: Pass")
        print("Grade : C")
    elif score >= 50:
        print("Result: Pass")
```

```
    print("Grade : D")
else:
    print("Result: Fail")
    print("Grade : F")
else:
    print("Invalid input. Please enter numbers only.")
```

Expected Output

```
Enter your score: 85
Result: Pass
Grade : A
```

OR

```
Enter your score: 45
Result: Fail
Grade : F
```

OR

```
Enter your score: abc
Invalid input. Please enter numbers only.
```

Optional Exercise

Enhance the program to:

- display feedback message
- A = Excellent
- B = Good
- C = Satisfactory
- D = Needs improvement
- F = Repeat assessment

Answer

```
score_input = input("Enter your score: ")
```

```
if score_input.isdigit():
    score = int(score_input)

    if score < 0 or score > 100:
        print("Invalid score. Score must be between 0 and 100.")
    elif score >= 80:
        print("Result: Pass")
        print("Grade : A")
        print("Feedback: Excellent")
    elif score >= 70:
        print("Result: Pass")
        print("Grade : B")
        print("Feedback: Good")
    elif score >= 60:
        print("Result: Pass")
        print("Grade : C")
        print("Feedback: Satisfactory")
    elif score >= 50:
        print("Result: Pass")
        print("Grade : D")
        print("Feedback: Needs improvement")
    else:
        print("Result: Fail")
        print("Grade : F")
        print("Feedback: Repeat assessment")
else:
    print("Invalid input. Please enter numbers only.")
```

Key Takeaways

- `if` handles the first condition
- `elif` checks multiple conditions
- `else` handles remaining cases
- Validation prevents invalid scores
- Decision apps use rules to produce results
- Grading systems are simple business-rule examples

LAB 8 — Mini Decision Engine

Objective

By the end of this lab, you will be able to:

- Simulate a login system
- Validate user credentials
- Build a menu-based program
- Use nested `if` and decision logic
- Control program flow based on user choice



Step-by-Step with Code

Step 1 — Basic Login Input

```
username = input("Enter username: ")
password = input("Enter password: ")

print(username, password)
```

Step 2 — Validate Login

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "1234":
    print("Login successful")
else:
    print("Invalid credentials")
```

Step 3 — Add Menu After Login

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "1234":
    print("Login successful")

    print("\n=== Menu ===")
    print("1. View Profile")
    print("2. Logout")
else:
    print("Invalid credentials")
```

Step 4 — Handle Menu Selection

```
username = input("Enter username: ")
password = input("Enter password: ")
```

```

if username == "admin" and password == "1234":
    print("Login successful")

    print("\n=== Menu ===")
    print("1. View Profile")
    print("2. Logout")

    choice = input("Select option: ")

    if choice == "1":
        print("Profile: Admin User")
    elif choice == "2":
        print("Logging out...")
    else:
        print("Invalid option")
else:
    print("Invalid credentials")

```

Step 5 — Full Mini Decision Engine

```

username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "1234":
    print("\nLogin successful!")

    print("\n=== Menu ===")
    print("1. View Profile")
    print("2. Change Password")
    print("3. Logout")

    choice = input("Select option: ")

    if choice == "1":
        print("\nProfile:")
        print("Username: admin")
        print("Role: Administrator")

    elif choice == "2":
        new_pass = input("Enter new password: ")
        print("Password updated (simulation)")

```

```
elif choice == "3":  
    print("Logging out...")  
  
else:  
    print("Invalid menu option")  
else:  
    print("Invalid username or password")
```

Expected Output

```
Enter username: admin  
Enter password: 1234  
  
Login successful!  
  
=== Menu ===  
1. View Profile  
2. Change Password  
3. Logout  
  
Select option: 1  
  
Profile:  
Username: admin  
Role: Administrator
```

Optional Exercise

Enhance the system to:

- allow 3 login attempts
- block access after failed attempts
- loop menu until user logs out

Answer

```
attempts = 0  
  
while attempts < 3:
```

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "1234":
    print("\nLogin successful!")

    while True:
        print("\n=== Menu ===")
        print("1. View Profile")
        print("2. Logout")

        choice = input("Select option: ")

        if choice == "1":
            print("Profile: Admin User")
        elif choice == "2":
            print("Logging out...")
            break
        else:
            print("Invalid option")
            break
    else:
        print("Invalid credentials")
        attempts += 1

if attempts == 3:
    print("Too many failed attempts. Access blocked.")
```

Key Takeaways

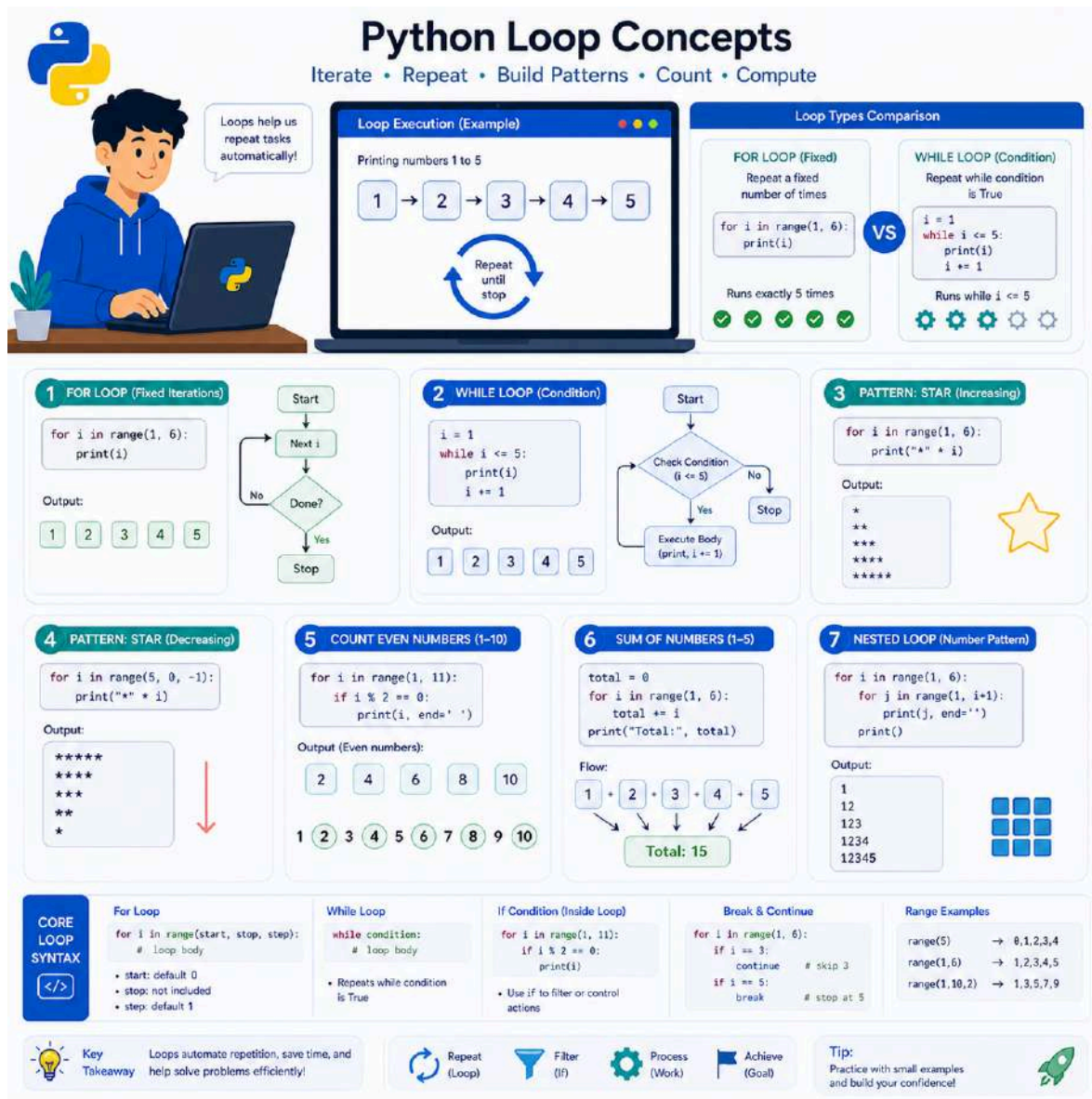
- Combine login + menu = decision system
- Use **and** for credential validation
- Nested **if** controls program flow
- Menu systems are common in real apps
- Loops improve user interaction
- This is a foundation for real systems (APIs, apps, dashboards)

LAB 9 — Loop Exercises

Objective

By the end of this lab, you will be able to:

- Use **for** and **while** loops
- Build pattern-based output
- Implement counting logic
- Combine loops with conditions
- Strengthen iteration skills



Python Loop Concepts
Iterate • Repeat • Build Patterns • Count • Compute

Loops help us repeat tasks automatically!

Loop Execution (Example)
Printing numbers 1 to 5
1 → 2 → 3 → 4 → 5
Repeat until stop

Loop Types Comparison

FOR LOOP (Fixed)
Repeat a fixed number of times
`for i in range(1, 6):
 print(i)`
Runs exactly 5 times

VS

WHILE LOOP (Condition)
Repeat while condition is True
`i = 1
while i <= 5:
 print(i)
 i += 1`
Runs while i <= 5

1 FOR LOOP (Fixed Iterations)
`for i in range(1, 6):
 print(i)`
Output: 1 2 3 4 5

2 WHILE LOOP (Condition)
`i = 1
while i <= 5:
 print(i)
 i += 1`
Output: 1 2 3 4 5

3 PATTERN: STAR (Increasing)
`for i in range(1, 6):
 print("*" * i)`
Output: *
**

4 PATTERN: STAR (Decreasing)
`for i in range(5, 0, -1):
 print("*" * i)`
Output: *****

**
*

5 COUNT EVEN NUMBERS (1-10)
`for i in range(1, 11):
 if i % 2 == 0:
 print(i, end=' ')`
Output (Even numbers): 2 4 6 8 10

6 SUM OF NUMBERS (1-5)
`total = 0
for i in range(1, 6):
 total += i
print("Total:", total)`
Flow: 1 + 2 + 3 + 4 + 5
Total: 15

7 NESTED LOOP (Number Pattern)
`for i in range(1, 6):
 for j in range(1, i+1):
 print(j, end='')
 print()`
Output: 1
12
123
1234
12345

CORE LOOP SYNTAX

For Loop
`for i in range(start, stop, step):
 # loop body`
• start: default 0
• stop: not included
• step: default 1

While Loop
`while condition:
 # loop body`
• Repeats while condition is True

If Condition (Inside Loop)
`for i in range(1, 11):
 if i % 2 == 0:
 print(i)`
• Use if to filter or control actions

Break & Continue
`for i in range(1, 6):
 if i == 3:
 continue # skip 3
 if i == 5:
 break # stop at 5`

Range Examples
`range(5)` → 0,1,2,3,4
`range(1,6)` → 1,2,3,4,5
`range(1,10,2)` → 1,3,5,7,9

Key Takeaway
Loops automate repetition, save time, and help solve problems efficiently!

Repeat (Loop) **Filter (If)** **Process (Work)** **Achieve (Goal)**

Tip:
Practice with small examples and build your confidence!

Step-by-Step with Code

Step 1 — Basic Counting (For Loop)

```
for i in range(1, 6):  
    print(i)
```

✓ Output:

```
1  
2  
3  
4  
5
```

```
count = 1  
  
while count <= 5:  
    print(count)  
    count += 1
```

Step 3 — Pattern Printing (Stars)

```
for i in range(1, 6):  
    print("'" * i)
```

✓ Output:

```
*  
**  
***  
****  
*****
```

Step 4 — Reverse Pattern

```
for i in range(5, 0, -1):  
    print("*" * i)
```

✓ Output:

```
*****  
****  
***  
**  
*
```

Step 5 — Counting Even Numbers

```
for i in range(1, 11):  
    if i % 2 == 0:  
        print(i)
```

✓ Output:

```
2  
4  
6  
8  
10
```

Step 6 — Total Sum (Counting Program)

```
total = 0  
  
for i in range(1, 6):  
    total += i
```



```
print("Total:", total)
```

✓ Output:

```
Total: 15
```

Step 7 — Full Program (Pattern + Counting)

```
print("=== Pattern ===")
for i in range(1, 6):
    print(" " * i)

print("\n=== Counting ===")
total = 0

for i in range(1, 11):
    total += i

print("Sum from 1 to 10:", total)
```

Optional Exercise

Enhance the program to:

- print number pattern (1, 12, 123, ...)
- count odd numbers
- display total of odd numbers

Answer

```
print("=== Number Pattern ===")
for i in range(1, 6):
    for j in range(1, i + 1):
        print(j, end="")
    print()

print("\n=== Odd Numbers ===")
total = 0
```

```
for i in range(1, 11):  
    if i % 2 != 0:  
        print(i)  
        total += i  
  
print("Total odd numbers:", total)
```

Key Takeaways

- Loops automate repetition
- **for** is best for fixed iterations
- **while** is best for condition-based loops
- Patterns help understand loop behavior
- Counting is a common real-world use case
- Combining loops + conditions builds logic

LAB 10 — Dice Roll Game

Objective

By the end of this lab, you will be able to:

- Generate random numbers
- Simulate a dice roll (1–6)
- Use loops for repeated gameplay
- Control program flow using conditions
- Build a simple interactive game



Python Dice Roll Game
Randomness • Loop • Decision • Repeat

Let's roll the dice!

Random Number Flow

1 Roll Dice
Generate a random number
`import random
roll = random.randint(1, 6)`

2 Show Result
Display the rolled value
You rolled: 3

3 Decision (If/Else)
Check the outcome
if roll == 6: Jackpot! 🎉
elif roll == 1: Bad luck 😞
else: Keep trying! 🍀

4 Ask Again
Loop until user quits
Roll again? (y/n)
y

5 Stats
Track game statistics
Total Rolls: 10
Jackpot (6s): 2
Ones: 1
Session Active

Python Code (Simplified)

```
import random
total = 0
jackpot = 0
while True:
    roll = random.randint(1, 6)
    total += 1
    print(f"You rolled: {roll}")
    if roll == 6:
        jackpot += 1
        print("🎉 Jackpot!")
    elif roll == 1:
        print("😞 Bad luck")
    else:
        print("🍀 Keep trying!")
    again = input("Roll again? (y/n): ")
    if again.lower() == 'n':
        break
print(f"Total rolls: {total}, Jackpots: {jackpot}")
```

Game Flow

Start → Roll Dice (1-6) → Show Result → Check Value → 6: Jackpot! | 1: Bad luck | 2-5: Keep trying → Ask Again? (y/n) → End

Sample Run

```
You rolled: 4
Keep trying!
Roll again? (y/n): y
You rolled: 1
Bad luck 😞
Roll again? (y/n): y
You rolled: 6
Jackpot! 🎉
Roll again? (y/n): n
Total rolls: 3, Jackpots: 1
```

Key Concepts

- Random Number
- Loop (while True)
- Condition (if / elif / else)
- User Input (y/n)
- Repeat Cycle
- Statistics Tracking

Step-by-Step with Code

Step 1 — Import Random Module

```
import random
```

Step 2 — Generate Random Number

```
import random

roll = random.randint(1, 6)
print("Dice roll:", roll)
```

✓ Output (example):

```
Dice roll: 4
```

Step 3 — Roll Dice Multiple Times (Loop)

```
import random

for i in range(5):
    roll = random.randint(1, 6)
    print("Roll:", roll)
```

Step 4 — Add User Control (Play Again)

```
import random

while True:
    roll = random.randint(1, 6)
    print("Dice roll:", roll)
```

```
choice = input("Roll again? (y/n): ")

if choice == "n":
    break
```

Step 5 — Add Game Message

```
import random

while True:
    roll = random.randint(1, 6)

    print("Dice roll:", roll)

    if roll == 6:
        print("🎉 Jackpot! You rolled a 6!")
    elif roll == 1:
        print("😞 Bad luck!")
    else:
        print("Try again!")

    choice = input("Roll again? (y/n): ")

    if choice == "n":
        print("Game ended.")
        break
```

Step 6 — Full Dice Game

```
import random

print("=== Dice Roll Game ===")

while True:
    roll = random.randint(1, 6)
```

```
print("\nYou rolled:", roll)

if roll == 6:
    print("🎉 Jackpot! You win!")
elif roll == 1:
    print("😞 Bad luck!")
else:
    print("Keep trying!")

choice = input("Roll again? (y/n): ").lower()

if choice == "n":
    print("\nThanks for playing!")
    break
```

Expected Output

```
=== Dice Roll Game ===
```

```
You rolled: 3
Keep trying!
Roll again? (y/n): y
```

```
You rolled: 6
🎉 Jackpot! You win!
Roll again? (y/n): n
```

```
Thanks for playing!
```

Optional Exercise

Enhance the game to:

- keep track of total rolls
- count how many times user gets 6
- display statistics at the end

Answer

```

import random

print("=== Dice Roll Game ===")

total_rolls = 0
jackpot_count = 0

while True:
    roll = random.randint(1, 6)
    total_rolls += 1

    print("\nYou rolled:", roll)

    if roll == 6:
        print("🎉 Jackpot! You win!")
        jackpot_count += 1
    elif roll == 1:
        print("😞 Bad luck!")
    else:
        print("Keep trying!")

    choice = input("Roll again? (y/n): ").lower()

    if choice == "n":
        print("\n=== Game Stats ===")
        print("Total Rolls:", total_rolls)
        print("Total Jackpot (6s):", jackpot_count)
        print("Thanks for playing!")
        break

```

Key Takeaways

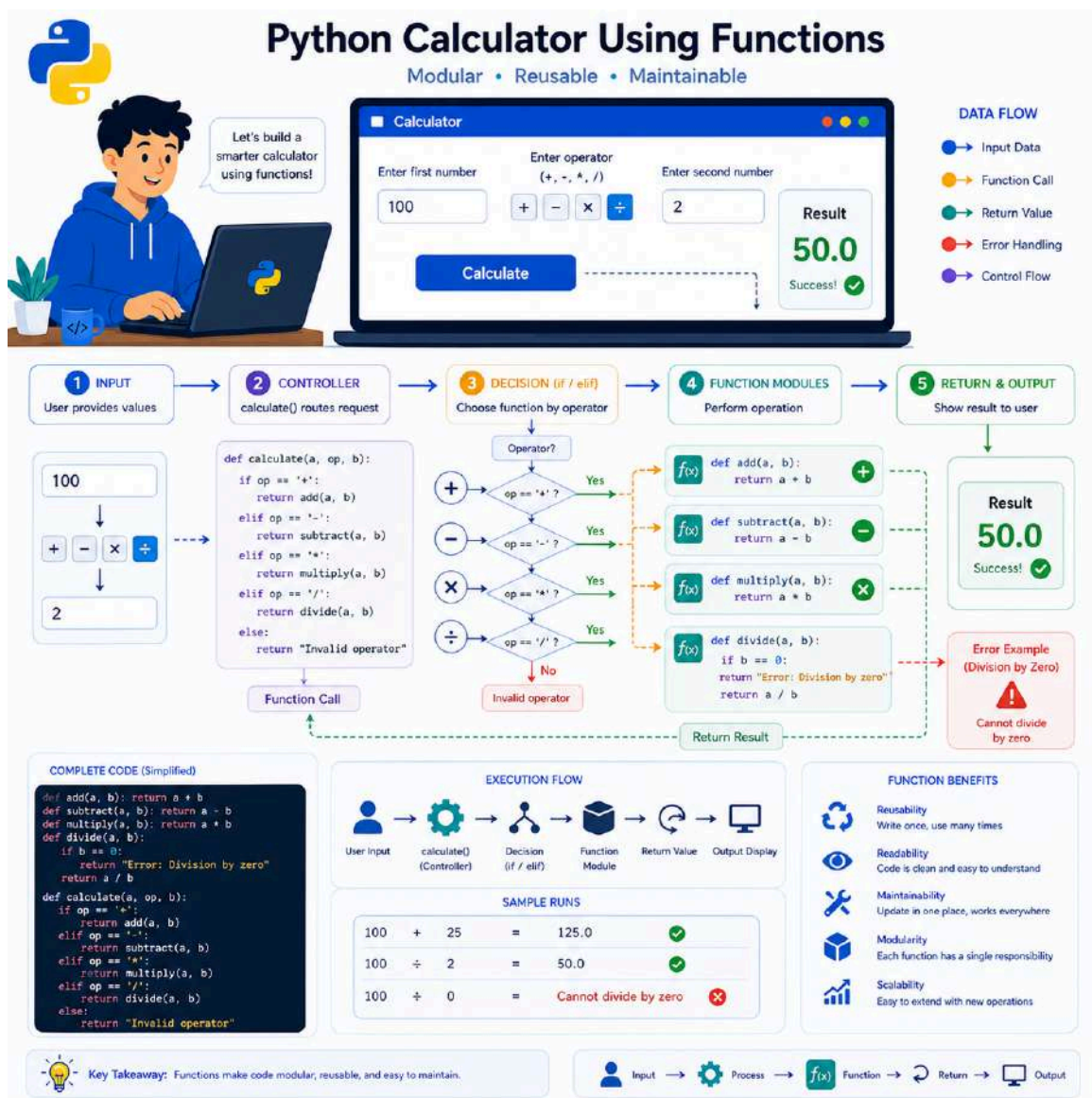
- `random.randint()` generates random values
- Loops enable continuous gameplay
- `while True` creates game loop
- `break` exits loop
- Conditions create game logic
- Combining randomness + loops = simulation systems

LAB 11 — Function-Based Calculator

Objective

By the end of this lab, you will be able to:

- Create reusable functions
- Pass values using parameters
- Return calculation results
- Organize calculator logic cleanly
- Reduce repeated code



Step-by-Step with Code

Step 1 — Create Addition Function

```
def add(a, b):  
    return a + b  
  
result = add(10, 5)  
print("Result:", result)
```

✓ Output:

```
Result: 15
```

Step 2 — Create Basic Operation Functions

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    return a / b
```

Step 3 — Get User Input

```
num1 = float(input("Enter first number: "))  
operator = input("Enter operator (+, -, *, /): ")  
num2 = float(input("Enter second number: "))
```

Step 4 — Connect Functions with Decision Logic

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    return a / b  
  
num1 = float(input("Enter first number: "))  
operator = input("Enter operator (+, -, *, /): ")  
num2 = float(input("Enter second number: "))  
  
if operator == "+":  
    result = add(num1, num2)  
elif operator == "-":  
    result = subtract(num1, num2)  
elif operator == "*":  
    result = multiply(num1, num2)  
elif operator == "/":  
    result = divide(num1, num2)  
else:  
    result = "Invalid operator"  
  
print("Result:", result)
```

Step 5 — Handle Division by Zero

```
def divide(a, b):  
    if b == 0:  
        return "Cannot divide by zero"  
    return a / b
```

Step 6 — Full Function-Based Calculator

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Cannot divide by zero"
    return a / b

print("=== Function-Based Calculator ===")

num1 = float(input("Enter first number: "))
operator = input("Enter operator (+, -, *, /): ")
num2 = float(input("Enter second number: "))

if operator == "+":
    result = add(num1, num2)
elif operator == "-":
    result = subtract(num1, num2)
elif operator == "*":
    result = multiply(num1, num2)
elif operator == "/":
    result = divide(num1, num2)
else:
    result = "Invalid operator"

print("Result:", result)
```

Expected Output

```
=== Function-Based Calculator ===  
Enter first number: 10  
Enter operator (+, -, *, /): *  
Enter second number: 5  
Result: 50.0
```

OR

```
=== Function-Based Calculator ===  
Enter first number: 10  
Enter operator (+, -, *, /): /  
Enter second number: 0  
Result: Cannot divide by zero
```

Optional Exercise

Enhance the calculator to:

- support modulus `%`
- support exponent `**`
- place operation decision inside a function called `calculate()`

Answer

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b  
  
def multiply(a, b):  
    return a * b  
  
def divide(a, b):
```

```

    if b == 0:
        return "Cannot divide by zero"
    return a / b

def modulus(a, b):
    if b == 0:
        return "Cannot use modulus with zero"
    return a % b

def exponent(a, b):
    return a ** b

def calculate(a, operator, b):
    if operator == "+":
        return add(a, b)
    elif operator == "-":
        return subtract(a, b)
    elif operator == "*":
        return multiply(a, b)
    elif operator == "/":
        return divide(a, b)
    elif operator == "%":
        return modulus(a, b)
    elif operator == "**":
        return exponent(a, b)
    else:
        return "Invalid operator"

print("=== Advanced Function-Based Calculator ===")

num1 = float(input("Enter first number: "))
operator = input("Enter operator (+, -, *, /, %, **): ")
num2 = float(input("Enter second number: "))

result = calculate(num1, operator, num2)

print("Result:", result)

```

Key Takeaways

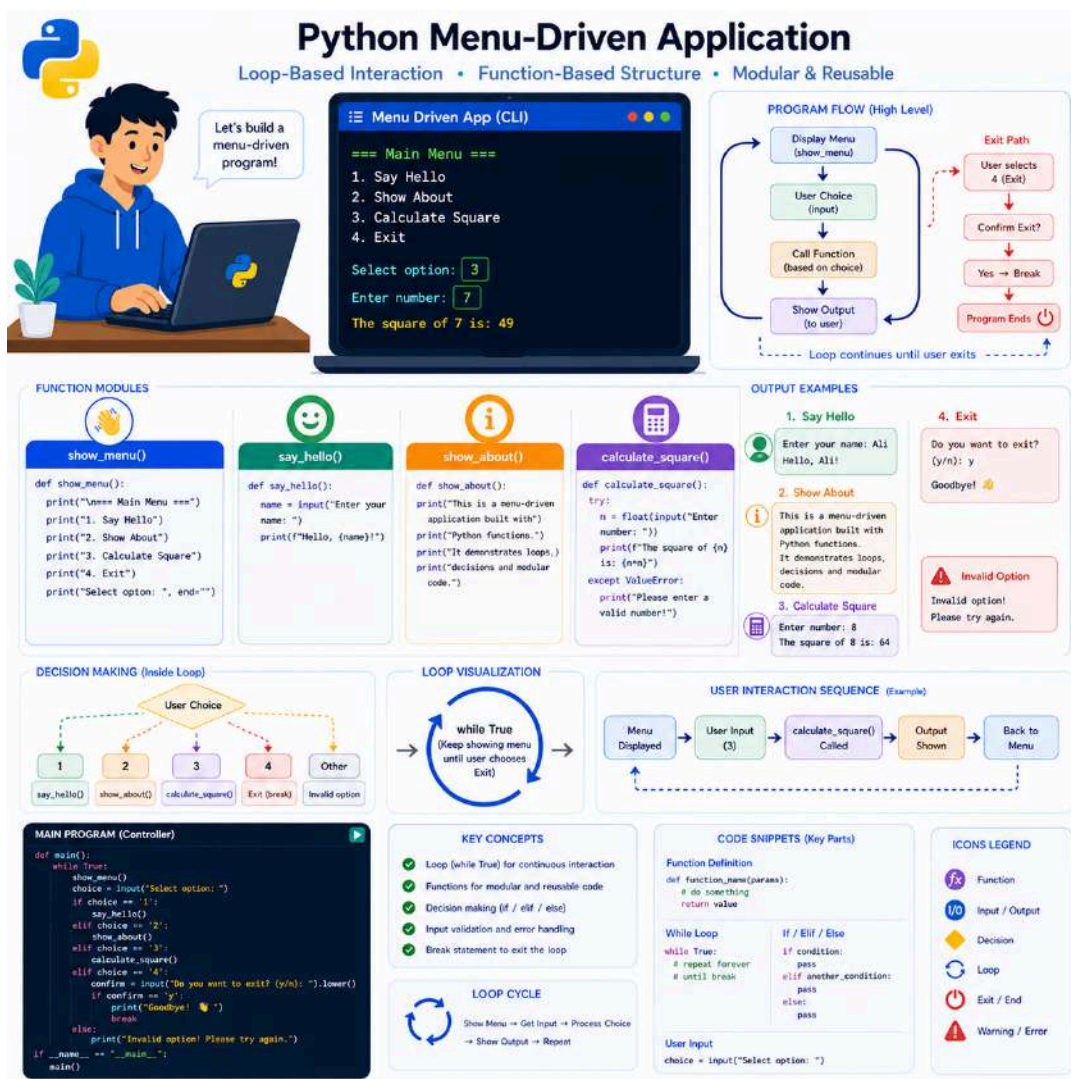
- Functions make code reusable
- Parameters pass values into functions
- `return` sends results back
- Calculator logic becomes easier to maintain
- Error handling can be placed inside functions
- Reusable functions prepare code for larger applications

LAB 12 — Menu System

Objective

By the end of this lab, you will be able to:

- Build a menu-driven program
- Use loops to repeat actions
- Use functions to organize code
- Handle user menu selection
- Exit program safely



Step-by-Step with Code

Step 1 — Create Menu Display Function

```
def show_menu():  
    print("\n=== Main Menu ===")  
    print("1. Say Hello")  
    print("2. Show About")  
    print("3. Exit")
```

Step 2 — Call the Menu Function

```
def show_menu():  
    print("\n=== Main Menu ===")  
    print("1. Say Hello")  
    print("2. Show About")  
    print("3. Exit")
```

```
show_menu()
```

Step 3 — Get User Choice

```
def show_menu():  
    print("\n=== Main Menu ===")  
    print("1. Say Hello")  
    print("2. Show About")  
    print("3. Exit")
```

```
show_menu()  
choice = input("Select option: ")  
  
print("You selected:", choice)
```

Step 4 — Add Menu Actions


```
def show_menu():
    print("\n=== Main Menu ===")
    print("1. Say Hello")
    print("2. Show About")
    print("3. Exit")

def say_hello():
    print("Hello! Welcome to the system.")

def show_about():
    print("This is a simple Python menu system.")

show_menu()
choice = input("Select option: ")

if choice == "1":
    say_hello()
elif choice == "2":
    show_about()
elif choice == "3":
    print("Exiting program...")
else:
    print("Invalid option")
```

Step 5 — Add Loop to Keep Menu Running

```
def show_menu():
    print("\n=== Main Menu ===")
    print("1. Say Hello")
    print("2. Show About")
    print("3. Exit")

def say_hello():
    print("Hello! Welcome to the system.")
```

```
def show_about():
    print("This is a simple Python menu system.")

while True:
    show_menu()
    choice = input("Select option: ")

    if choice == "1":
        say_hello()
    elif choice == "2":
        show_about()
    elif choice == "3":
        print("Exiting program...")
        break
    else:
        print("Invalid option")
```

Step 6 — Full Menu System

```
def show_menu():
    print("\n=== Main Menu ===")
    print("1. Say Hello")
    print("2. Show About")
    print("3. Calculate Square")
    print("4. Exit")

def say_hello():
    name = input("Enter your name: ")
    print(f"Hello, {name}!")

def show_about():
    print("This application demonstrates loops and functions.")

def calculate_square():
    number_input = input("Enter a number: ")

    if number_input.isdigit():
        number = int(number_input)
```

```

        print("Square:", number * number)
    else:
        print("Invalid input. Please enter a number.")

while True:
    show_menu()

    choice = input("Select option: ")

    if choice == "1":
        say_hello()
    elif choice == "2":
        show_about()
    elif choice == "3":
        calculate_square()
    elif choice == "4":
        print("Exiting program...")
        break
    else:
        print("Invalid option. Please try again.")

```

Expected Output

```

=== Main Menu ===
1. Say Hello
2. Show About
3. Calculate Square
4. Exit
Select option: 1
Enter your name: Ali
Hello, Ali!

OR

=== Main Menu ===
1. Say Hello
2. Show About
3. Calculate Square
4. Exit
Select option: 4
Exiting program...

```

Optional Exercise

Enhance the menu system to:

- add calculator option
- add repeat confirmation before exit
- validate empty input

Answer

```
def show_menu():
    print("\n=== Main Menu ===")
    print("1. Say Hello")
    print("2. Show About")
    print("3. Calculator")
    print("4. Exit")

def say_hello():
    name = input("Enter your name: ").strip()

    if name == "":
        print("Name cannot be empty.")
    else:
        print(f"Hello, {name.title()}!")

def show_about():
    print("This is a function-based Python menu application.")

def calculator():
    num1_input = input("Enter first number: ")
    operator = input("Enter operator (+, -, *, /): ")
    num2_input = input("Enter second number: ")

    try:
        num1 = float(num1_input)
        num2 = float(num2_input)

        if operator == "+":
            print("Result:", num1 + num2)
        elif operator == "-":
```

```

        print("Result:", num1 - num2)
    elif operator == "**":
        print("Result:", num1 * num2)
    elif operator == "/":
        if num2 == 0:
            print("Cannot divide by zero.")
        else:
            print("Result:", num1 / num2)
    else:
        print("Invalid operator.")

except:
    print("Invalid number input.")

while True:
    show_menu()

    choice = input("Select option: ").strip()

    if choice == "1":
        say_hello()
    elif choice == "2":
        show_about()
    elif choice == "3":
        calculator()
    elif choice == "4":
        confirm = input("Are you sure you want to exit? (y/n): ").lower()

        if confirm == "y":
            print("Exiting program...")
            break
        else:
            print("Returning to menu...")
    else:
        print("Invalid option. Please try again.")

```

Key Takeaways

- Menu systems use loops to keep running

- Functions organize program features
- `while True` creates continuous interaction
- `break` exits the menu safely
- Input validation improves reliability
- This structure prepares for CLI applications

Lab 13: List Management System

Objective

By the end of this lab, you will be able to:

- Create and manage lists
- Add items dynamically
- Remove items safely
- Use loops for continuous interaction
- Build a simple CLI management system

Step-by-Step with Code

Step 1 — Create an Empty List

```
items = []  
  
print(items)
```

Step 2 — Add Items to List

```
items = []  
  
item = input("Enter item to add: ")  
items.append(item)  
  
print(items)
```

Step 3 — Remove Items from List

```
items = ["apple", "banana", "orange"]  
  
item = input("Enter item to remove: ")  
  
if item in items:
```

```
    items.remove(item)
    print("Item removed")
else:
    print("Item not found")

print(items)
```

Step 4 — Display All Items

```
items = ["apple", "banana", "orange"]

print("\n=== Item List ===")
for item in items:
    print("-", item)
```

Step 5 — Build Menu System

```
items = []

while True:
    print("\n=== List Menu ===")
    print("1. Add Item")
    print("2. Remove Item")
    print("3. View Items")
    print("4. Exit")

    choice = input("Select option: ")

    if choice == "1":
        item = input("Enter item: ")
        items.append(item)
        print("Item added")

    elif choice == "2":
        item = input("Enter item to remove: ")

        if item in items:
            items.remove(item)
```



```
        print("Item removed")
    else:
        print("Item not found")

    elif choice == "3":
        print("\nItems:")
        for i in items:
            print("-", i)

    elif choice == "4":
        print("Exiting...")
        break

    else:
        print("Invalid option")
```

Expected Output

```
=== List Menu ===
1. Add Item
2. Remove Item
3. View Items
4. Exit
Select option: 1
Enter item: apple
Item added

Select option: 3
Items:
- apple
```

Optional Exercise

Enhance the system to:

- prevent duplicate items
- show item count
- clear all items

Answer

```
items = []

while True:
    print("\n=== List Menu ===")
    print("1. Add Item")
    print("2. Remove Item")
    print("3. View Items")
    print("4. Clear List")
    print("5. Exit")

    choice = input("Select option: ")

    if choice == "1":
        item = input("Enter item: ").strip()

        if item == "":
            print("Item cannot be empty")
        elif item in items:
            print("Item already exists")
        else:
            items.append(item)
            print("Item added")

    elif choice == "2":
        item = input("Enter item to remove: ")

        if item in items:
            items.remove(item)
            print("Item removed")
        else:
            print("Item not found")

    elif choice == "3":
        print("\nItems:")
        print("Total:", len(items))
        for i in items:
            print("-", i)

    elif choice == "4":
        items.clear()
        print("List cleared")
```

```
elif choice == "5":  
    print("Exiting...")  
    break  
  
else:  
    print("Invalid option")
```

Key Takeaways

- Lists store dynamic data
- `.append()` adds items
- `.remove()` deletes items
- Use `in` to check existence
- Loops enable continuous systems
- Menu systems simulate real applications

LAB 14 — Set Processing

Objective

By the end of this lab, you will be able to:

- Create a set
- Convert list to set
- Remove duplicate values
- Compare list vs set behavior
- Build simple data-cleaning logic



Python Data Cleaning with Sets
Remove Duplicates • Clean Text • Get Unique Values

Let's clean this data!

Python Data Cleaning (Sets)

Enter names separated by comma:
ali, siti, ali, john, siti

Processing...
Cleaning complete! ✓

What is a Set?

- ✓ A set stores unique items.
- ✓ Duplicates are automatically removed.
- ✓ Unordered collection of elements.
- ✓ Great for data cleaning!

1 INPUT
Raw string from user
ali, siti, ali, john, siti

2 SPLIT
Split string into list
ali, siti, ali, john, siti

3 CLEAN
Trim spaces & Capitalize
Ali, Siti, Ali, John, Siti

4 CONVERT TO SET
Remove duplicates (Keep unique values)
Ali, Siti, John

5 UNIQUE OUTPUT
Back to list (optional) for display
=== Cleaned Names ===
Ali
Siti
John ✓

LIST (with duplicates)
Ali
Siti
Ali
John
Siti

VS

SET (unique only)
Ali
Siti
John

Sets store only unique values. Duplicates are removed. ✓

SUMMARY

Original Count	Unique Count	Duplicates Removed	Duplicate Items
5	3	2	ali, siti

1 Get Input
data = input("Enter names separated by comma: ")

2 Split into List
names_list = [name for name in data.split(',') if name.strip()]

3 Clean the Data
clean_list = [name.strip().title() for name in names_list]

4 Convert to Set
unique_set = set(clean_list)

5 Back to List (Optional)
unique_list = list(unique_set)

Full Example Code

```
1 data = input("Enter names separated by comma: ")
2 names_list = [name for name in data.split(',') if name.strip()]
3 clean_list = [name.strip().title() for name in names_list]
4 unique_set = set(clean_list)
5 unique_list = list(unique_set)
6 print("\n=== Cleaned Names ===")
7 for name in unique_list:
8     print(name)
```

Sample Run

```
Enter names separated by comma:
ali, siti, ali, john, siti
=== Cleaned Names ===
Ali
Siti
John
Original Count : 5
Unique Count   : 3
Duplicates Removed : 2
```

KEY TAKEAWAYS

- ✓ Use split() to break string into list.
- ✓ Clean data using strip() and title().
- ✓ Convert list to set to remove duplicates.
- ✓ Convert set back to list for display.
- ✓ Sets are powerful for data cleaning!

Step-by-Step with Code

Step 1 — Create a List with Duplicates

```
names = ["Ali", "Siti", "Ali", "John", "Siti"]  
  
print(names)
```

Step 2 — Convert List to Set

```
names = ["Ali", "Siti", "Ali", "John", "Siti"]  
  
unique_names = set(names)  
  
print(unique_names)
```

✓ Output:

```
{'Ali', 'Siti', 'John'}
```

Step 3 — Convert Set Back to List

```
names = ["Ali", "Siti", "Ali", "John", "Siti"]  
  
unique_names = list(set(names))  
  
print(unique_names)
```

Step 4 — Accept User Input

```
raw_input = input("Enter names separated by comma: ")  
  
names = raw_input.split(",")
```

```
print(names)
```

Step 5 — Clean Spaces

```
raw_input = input("Enter names separated by comma: ")

names = raw_input.split(",")

clean_names = []

for name in names:
    clean_names.append(name.strip().title())

print(clean_names)
```

Step 6 — Full Duplicate Removal Program

```
raw_input = input("Enter names separated by comma: ")

names = raw_input.split(",")

clean_names = []

for name in names:
    clean_names.append(name.strip().title())

unique_names = list(set(clean_names))

print("\n=== Cleaned Names ===")
for name in unique_names:
    print("-", name)
```

Expected Output

```
Enter names separated by comma: ali, siti, ali, john, siti

=== Cleaned Names ===
```

- Ali
- Siti
- John

Optional Exercise

Enhance the program to:

- show total original names
- show total unique names
- show how many duplicates were removed

Answer

```
raw_input = input("Enter names separated by comma: ")

names = raw_input.split(",")

clean_names = []

for name in names:
    clean_names.append(name.strip().title())

unique_names = list(set(clean_names))

duplicates_removed = len(clean_names) - len(unique_names)

print("\n=== Duplicate Removal Summary ===")
print("Original Count:", len(clean_names))
print("Unique Count:", len(unique_names))
print("Duplicates Removed:", duplicates_removed)

print("\n=== Unique Names ===")
for name in unique_names:
    print("-", name)
```

Key Takeaways

- Sets automatically remove duplicates
- Lists keep duplicate values

- `set()` converts data into unique values
- `.split()` separates text into list items
- `.strip()` cleans extra spaces
- Sets are useful for data cleaning and validation

LAB 15 — Dictionary App

Objective

By the end of this lab, you will be able to:

- Create and use dictionaries
- Store structured student data
- Add, update, and delete records
- Retrieve data using keys



Python Student Database Using Dictionaries

Store • Manage • Search • Update • Remove



```
Student Database (CLI)

=== Student Database ===

1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Exit

Select option: 
```



DICTIONARY DATA STRUCTURE

Key → Value pairs

Key (Name)	Value (Age)
Ali	20
Siti	19
John	21

Stored as: `students = {name: age}`

1 Add Student

Input: Name, Age

Name: Aisyah
Age: 22

Add

Dictionary Updated

Key (Name)	Value (Age)
Ali	20
Siti	19
John	21
Aisyah	22

3 Search Student

Input: Name

Search: Siti

Search

Found!

Siti → 19

Not Found!
Search: Zara
Student not found!

4 Remove Student

Input: Name

Remove: John

Remove

Dictionary Updated

Key (Name)	Value (Age)
Ali	20
Siti	19
Aisyah	22

+ Update Student (Optional)

Input: Name, New Age

Name: Ali
New Age: 21

Update

Dictionary Updated

Key (Name)	Value (Age)
Ali	21
Siti	19
Aisyah	22

2 View Students

Display All Students

Student List

- Ali - 21
- Siti - 19
- Aisyah - 22

Total Students: 3

VALIDATION & ERROR HANDLING

- Duplicate Name**
Ali already exists!
Cannot add duplicate.
- Student Not Found**
Zara not found in database!
Please try again.
- Invalid Age**
Age must be a positive number!

PYTHON DICTIONARY OPERATIONS (Cheat Sheet)

Add / Update

```
students[name] = age
# Add new or update existing student
```

View All

```
for name, age in students.items():
    print(f"{name} - {age}")
# items() returns (key, value) pairs
```

Search

```
if name in students:
    print(students[name])
else:
    print("Student not found!")
```

Remove

```
if name in students:
    students.pop(name)
    print("Removed!")
else:
    print("Not found!")
```

SAMPLE DATABASE (Dictionary)

```
students = {
    "Ali": 20,
    "Siti": 19,
    "John": 21,
    "Aisyah": 22
}
```

MAIN PROGRAM (Simplified)

```
students = {}
while True:
    print("\nStudent Database")
    print("1. Add Student")
    print("2. View Students")
    print("3. Search Student")
    print("4. Remove Student")
    print("5. Exit")
    choice = input("Select option: ")
    if choice == "1":
        # Add Student
        name = input("Name: ")
        age = input("Age: ")
        if name in students:
            print("Duplicate Name! Cannot add duplicate.")
        else:
            students[name] = int(age)
            print("Student added successfully!")
    elif choice == "2":
        # View Students
        for name, age in students.items():
            print(f"{name} - {age}")
    elif choice == "3":
        # Search Student
        name = input("Search: ")
        if name in students:
            print(f"{name} - {students[name]}")
        else:
            print("Student not found!")
    elif choice == "4":
        # Remove Student
        name = input("Remove: ")
        if name in students:
            students.pop(name)
            print("Student removed!")
        else:
            print("Student not found!")
    elif choice == "5":
        print("Goodbye!")
        break
    else:
        print("Invalid option! Try again.")
```

EXAMPLE RUN (Sample Interaction)

```
Student Database
1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Exit
Select option: 1
Name: Ali
Age: 20
Student added successfully!

Student Database
1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Exit
Select option: 2
Student List:
• Ali - 20
Total Students: 1

Student Database
1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Exit
Select option: 5
Goodbye!
```

KEY TAKEAWAYS

- ✓ Dictionaries store data as key-value pairs.
- ✓ Keys must be unique, values can repeat.
- ✓ Use `dict[key] = value` to add or update.
- ✓ Use `.items()` to iterate through all pairs.
- ✓ Use `.pop(key)` to remove a key.
- ✓ Great for storing and managing structured data!

Step-by-Step with Code

Step 1 — Create Dictionary

```
students = {}  
  
print(students)
```

Step 2 — Add Student Data

```
students = {}  
  
name = input("Enter student name: ")  
age = input("Enter student age: ")  
  
students[name] = age  
  
print(students)
```

Step 3 — Add Multiple Students

```
students = {}  
  
for i in range(2):  
    name = input("Enter student name: ")  
    age = input("Enter student age: ")  
  
    students[name] = age  
  
print(students)
```

Step 4 — Access Student Data

```
students = {"Ali": 20, "Siti": 19}
```

```
name = input("Enter student name to search: ")

if name in students:
    print("Age:", students[name])
else:
    print("Student not found")
```

Step 5 — Remove Student

```
students = {"Ali": 20, "Siti": 19}

name = input("Enter student name to remove: ")

if name in students:
    students.pop(name)
    print("Student removed")
else:
    print("Student not found")

print(students)
```

Step 6 — Display All Students

```
students = {"Ali": 20, "Siti": 19}

print("\n=== Student List ===")

for name, age in students.items():
    print(name, "-", age)
```

Step 7 — Full Student Database System

```
students = {}

while True:
```

```

print("\n=== Student Database ===")
print("1. Add Student")
print("2. View Students")
print("3. Search Student")
print("4. Remove Student")
print("5. Exit")

choice = input("Select option: ")

if choice == "1":
    name = input("Enter name: ")
    age = input("Enter age: ")

    students[name] = age
    print("Student added")

elif choice == "2":
    print("\nStudent List:")
    for name, age in students.items():
        print(name, "-", age)

elif choice == "3":
    name = input("Enter name to search: ")

    if name in students:
        print("Age:", students[name])
    else:
        print("Student not found")

elif choice == "4":
    name = input("Enter name to remove: ")

    if name in students:
        students.pop(name)
        print("Student removed")
    else:
        print("Student not found")

elif choice == "5":
    print("Exiting...")
    break

else:

```

```
print("Invalid option")
```

Expected Output

```
=== Student Database ===
```

1. Add Student
2. View Students
3. Search Student
4. Remove Student
5. Exit

```
Select option: 1  
Enter name: Ali  
Enter age: 20  
Student added
```

```
Select option: 2  
Student List:  
Ali - 20
```

Optional Exercise

Enhance the system to:

- prevent duplicate student names
- update student age
- display total number of students

Answer

```
students = {}
```

```
while True:  
    print("\n=== Student Database ===")  
    print("1. Add Student")  
    print("2. View Students")
```

```

print("3. Search Student")
print("4. Update Student")
print("5. Remove Student")
print("6. Exit")

choice = input("Select option: ")

if choice == "1":
    name = input("Enter name: ").strip().title()
    age = input("Enter age: ")

    if name in students:
        print("Student already exists")
    else:
        students[name] = age
        print("Student added")

elif choice == "2":
    print("\nStudent List (Total:", len(students), ")")
    for name, age in students.items():
        print(name, "-", age)

elif choice == "3":
    name = input("Enter name to search: ").strip().title()

    if name in students:
        print("Age:", students[name])
    else:
        print("Student not found")

elif choice == "4":
    name = input("Enter name to update: ").strip().title()

    if name in students:
        new_age = input("Enter new age: ")
        students[name] = new_age
        print("Student updated")
    else:
        print("Student not found")

elif choice == "5":
    name = input("Enter name to remove: ").strip().title()

```

```
if name in students:
    students.pop(name)
    print("Student removed")
else:
    print("Student not found")

elif choice == "6":
    print("Exiting...")
    break

else:
    print("Invalid option")
```

Key Takeaways

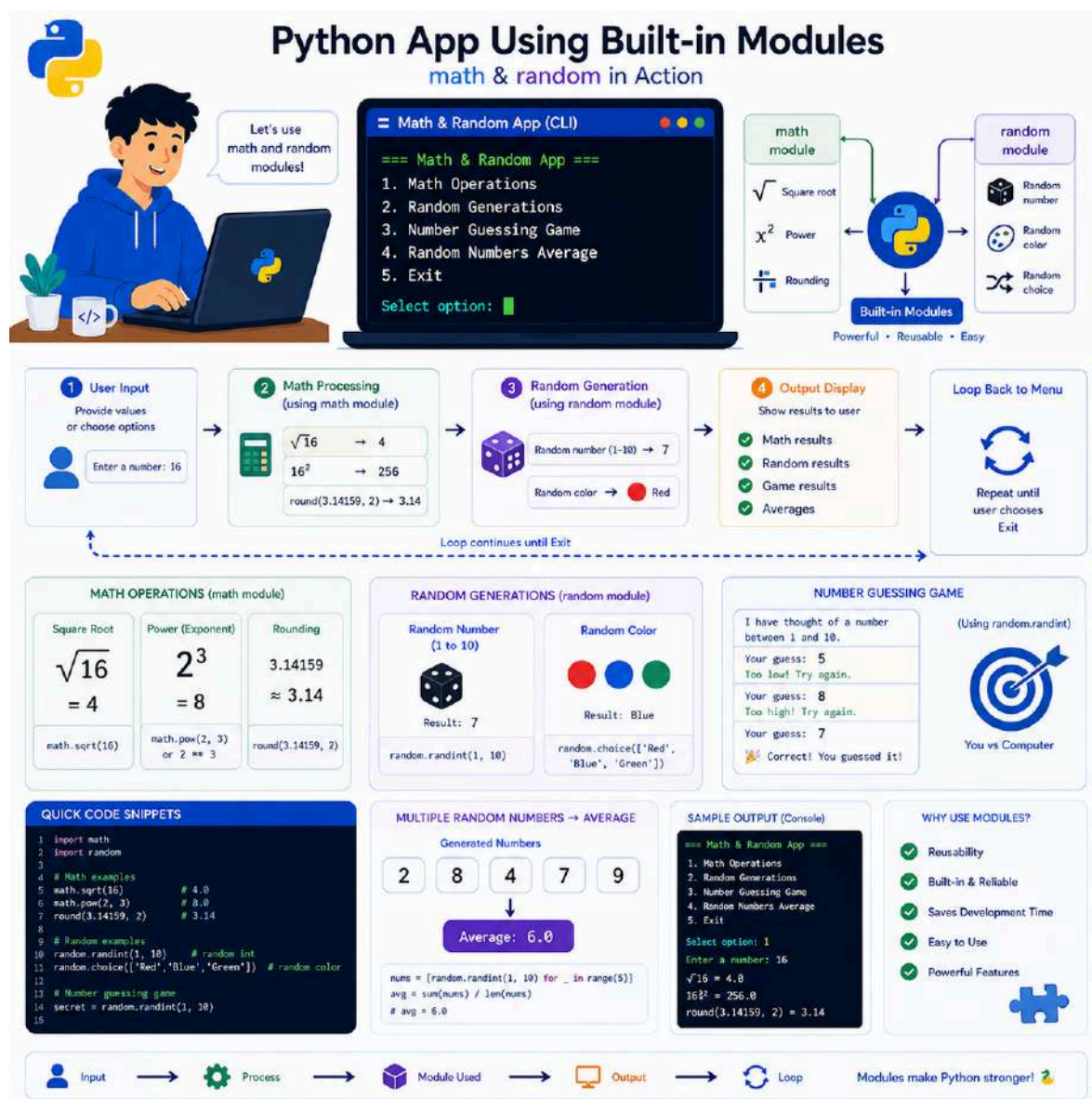
- Dictionaries store structured key-value data
- Keys must be unique (ideal for IDs/names)
- `.items()` helps iterate key-value pairs
- `.pop()` removes entries safely
- Menu systems simulate real database apps
- Foundation for backend/API development

LAB 16 — Module Usage

Objective

By the end of this lab, you will be able to:

- Import and use Python standard modules
- Perform mathematical operations using `math`
- Generate random values using `random`
- Combine modules in a small application
- Understand reusable built-in functionality



Step-by-Step with Code

Step 1 — Import Modules

```
import math
import random
```

Step 2 — Use Math Module

```
import math

print("Square root of 16:", math.sqrt(16))
print("Power (2^3):", math.pow(2, 3))
print("Ceiling of 4.2:", math.ceil(4.2))
```

Step 3 — Use Random Module

```
import random

print("Random number (1–10):", random.randint(1, 10))
```

Step 4 — Random Choice

```
import random

colors = ["red", "blue", "green"]

print("Random color:", random.choice(colors))
```

Step 5 — Build Simple Calculator with Math

```
import math

number = float(input("Enter a number: "))

print("Square root:", math.sqrt(number))
print("Square:", math.pow(number, 2))
```

Step 6 — Random Guessing Game

```
import random

number = random.randint(1, 5)

guess = int(input("Guess a number (1-5): "))

if guess == number:
    print("Correct!")
else:
    print("Wrong! The number was", number)
```

Step 7 — Full Module-Based Program

```
import math
import random

print("=== Math & Random App ===")

number = float(input("Enter a number: "))

print("\nMath Operations:")
print("Square root:", math.sqrt(number))
print("Square:", math.pow(number, 2))
print("Rounded up:", math.ceil(number))

print("\nRandom Features:")
print("Random number (1-10):", random.randint(1, 10))

colors = ["red", "blue", "green"]
```

```
print("Random color:", random.choice(colors))
```

Expected Output

```
=== Math & Random App ===  
Enter a number: 9  
  
Math Operations:  
Square root: 3.0  
Square: 81.0  
Rounded up: 9  
  
Random Features:  
Random number (1–10): 7  
Random color: blue
```

Optional Exercise

Enhance the program to:

- generate 5 random numbers
- calculate average of random numbers
- allow user to guess until correct

Answer

```
import math  
import random  
  
print("=== Advanced Math & Random App ===")  
  
# Generate random numbers  
numbers = []  
  
for i in range(5):  
    num = random.randint(1, 10)  
    numbers.append(num)  
  
print("\nRandom Numbers:", numbers)
```

```
# Calculate average
average = sum(numbers) / len(numbers)
print("Average:", average)

# Guessing game
target = random.randint(1, 5)

while True:
    guess = int(input("\nGuess number (1-5): "))

    if guess == target:
        print("Correct!")
        break
    else:
        print("Try again!")
```

Key Takeaways

- `import` allows use of built-in modules
- `math` provides calculation functions
- `random` generates random values
- Modules reduce need to write code from scratch
- Combine modules to build useful programs
- Foundation for advanced Python development

LAB 17 — Exception Handling

Objective

By the end of this lab, you will be able to:

- Use `try/except` to handle errors
- Prevent program crashes from invalid input
- Handle specific exceptions
- Provide user-friendly error messages
- Build safe input-processing logic



Python Exception Handling

Write Error-Safe Programs



Exception Handling Demo (CLI)

Enter a number: `abc`

Without Handling (Program Crashes)

```
ValueError: invalid literal for int() with base 10: 'abc'
Traceback (most recent call last):
  File "main.py", line 3, in <module>
    num = int(input("Enter a number: "))
ValueError: invalid literal for int() with base 10: 'abc'
```

With Handling (Safe Execution)

```
Enter a number: abc
Invalid input. Please enter a number.
Enter a number: 0
Cannot divide by zero.
Enter a number: 5
Square: 25
Program executed.
Total attempts: 3
```

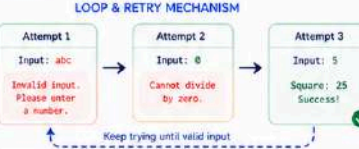
TRY-EXCEPT FLOW



EXCEPTION BRANCHES (Multiple Types)

<code>abc</code>	<code>ValueError</code> e.g. <code>int("abc")</code>	"Invalid number. Please enter a number."
<code>÷0</code>	<code>ZeroDivisionError</code> e.g. <code>10 / 0</code>	"Cannot divide by zero."
	Exception e.g. other errors	"Something went wrong. Please try again."

LOOP & RETRY MECHANISM



FINALLY BLOCK

Runs no matter what (success or error)

```
finally
    "Program executed."
```

OUTPUT SUMMARY

- ✓ Square: 25
- i Program executed.
- i Total attempts: 3

SIMPLE EXAMPLE CODE

```
1 import math
2
3 attempts = 0
4 while attempts < 5:
5     attempts += 1
6     try:
7         num = int(input("Enter a number: "))
8         result = num ** 2
9         print(f"Square: {result}")
10        break
11    except ValueError:
12        print("Invalid input. Please enter a number.")
13    except ZeroDivisionError:
14        print("Cannot divide by zero.")
15    except Exception as e:
16        print(f"An error occurred. Try again.")
17    finally:
18        print("Program executed.")
19 else:
20    print("Too many invalid attempts. Exiting...")
```

KEY CONCEPTS

- `</>` try: run risky code
- `except`: handle specific errors
- `finally`: always runs
- Handles errors without crashing
- Improves user experience
- Keeps program running safely

CODE SNIPPETS (CHEAT SHEET)

```
# Convert string to int safely
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("Invalid number.")

# Handle division by zero
try:
    result = 10 / num
except ZeroDivisionError:
    print("Cannot divide by zero.")

# Finally always runs
try:
    pass
except Exception:
    pass
finally:
    print("Program executed.")
```

ERROR-HANDLING BENEFITS

- ✓ Prevent program crashes
- ✓ Provide clear error messages
- ✓ Allow users to retry
- ✓ Handle different error types
- ✓ Ensure cleanup with finally

ATTEMPT LIMIT (OPTIONAL)

Max attempts: 5



Step-by-Step with Code

Step 1 — Problem Without Error Handling

```
num = int(input("Enter a number: "))  
print("Double:", num * 2)
```

✗ If user enters `abc` → program crashes

Step 2 — Basic try/except

```
try:  
    num = int(input("Enter a number: "))  
    print("Double:", num * 2)  
except:  
    print("Invalid input")
```

Step 3 — Handle Specific Exception

```
try:  
    num = int(input("Enter a number: "))  
    print("Double:", num * 2)  
except ValueError:  
    print("Please enter a valid number")
```

Step 4 — Multiple Exception Handling

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print("Result:", result)
except ValueError:
    print("Invalid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Step 5 — Add finally Block

```
try:
    num = int(input("Enter a number: "))
    print("Double:", num * 2)
except ValueError:
    print("Invalid input")
finally:
    print("Program executed")
```

Step 6 — Full Input Validation Program

```
while True:
    try:
        num = int(input("Enter a number: "))
        print("Square:", num * num)
        break
    except ValueError:
        print("Invalid input. Please enter a number.")
```

Expected Output

```
Enter a number: abc
Invalid input. Please enter a number.
Enter a number: 5
Square: 25
```

Optional Exercise

Enhance the program to:

- handle float input
- limit attempts to 3 tries
- display custom error message

Answer

```
attempts = 0
```



```
while attempts < 3:

    try:

        num = float(input("Enter a number: "))

        print("Square:", num * num)

        break

    except ValueError:

        print("Invalid input. Please enter a valid number.")

        attempts += 1

if attempts == 3:

    print("Too many failed attempts.")
```

Key Takeaways

- `try` handles risky code
- `except` catches errors
- Use specific exceptions when possible
- `finally` always executes
- Prevents program crashes
- Essential for real-world applications

LAB 18 — File Reader

Objective

By the end of this lab, you will be able to:

- Open and read text files
- Use `with open()` safely
- Read full content and line-by-line
- Handle missing file errors
- Build a simple file reader program



Step-by-Step with Code

Step 1 — Create a Text File

Create a file named:

```
data.txt
```

Add content:

```
Hello Python  
  
Welcome to file handling  
  
This is a sample file
```

Step 2 — Read Entire File

```
with open("data.txt", "r") as file:  
    content = file.read()  
    print(content)
```

Step 3 — Read Line by Line

```
with open("data.txt", "r") as file:  
    for line in file:  
        print(line.strip())
```

Step 4 — Read as List of Lines

```
with open("data.txt", "r") as file:

    lines = file.readlines()

print(lines)
```

Step 5 — Handle File Not Found Error

```
try:

    with open("data.txt", "r") as file:

        print(file.read())

except FileNotFoundError:

    print("File not found")
```

Step 6 — Full File Reader Program

```
filename = input("Enter file name: ")

try:

    with open(filename, "r") as file:

        print("\n=== File Content ===")
```

```
for line in file:

    print(line.strip())

except FileNotFoundError:

    print("File not found. Please check the file name.")
```

Expected Output

```
Enter file name: data.txt

=== File Content ===

Hello Python
Welcome to file handling
This is a sample file

OR

Enter file name: wrong.txt

File not found. Please check the file name.
```

Optional Exercise

Enhance the program to:

- count number of lines
- count number of words
- display total characters

Answer

```
filename = input("Enter file name: ")

try:

    with open(filename, "r") as file:

        lines = file.readlines()

        line_count = len(lines)

        word_count = 0

        char_count = 0

        for line in lines:

            word_count += len(line.split())

            char_count += len(line)

        print("\n=== File Summary ===")

        print("Lines:", line_count)

        print("Words:", word_count)

        print("Characters:", char_count)

except FileNotFoundError:

    print("File not found.")
```

Key Takeaways

- `open()` reads files

- "r" mode is for reading
- `with` ensures file is closed automatically
- `.read()` reads full content
- Loop reads line-by-line
- Exception handling prevents crash

LAB 19 — File Writer

Objective

By the end of this lab, you will be able to:

- Write data to a text file
- Use write ("w") and append ("a") modes
- Save user input permanently
- Format output for storage
- Build a simple data logging system

Python File Writing (Save Data to Text Files)

Collect Data → Write to File → Store Permanently

Flowchart:

- User Input:** Collect name and age from the user. (Name: Ali, Age: 20)
- Validate:** Check for valid input. (Name not empty, Age is a number, Age > 0)
- Format Data:** Prepare data to write. (name, age, timestamp)
- Write to File:** Write using file mode (w or a) to users.txt.
- Save Confirmation:** Show success message. (Data saved successfully!)
- Loop or Exit:** Add another? (y = continue, n = exit)

WRITE MODE ("w") - Overwrite: Existing file is cleared and new data is written.

APPEND MODE ("a") - Add New Data: New data is added to the end of the file.

FILE CONTENT (users.txt):

```
1 Ali,20,2024-05-27 10:15:30
2 Siti,19,2024-05-27 10:16:05
3 John,22,2024-05-27 10:16:42
```

VALIDATION & ERROR HANDLING:

- Empty Name: Name: "" (Name cannot be empty!)
- Invalid Age: Age: "abc" (Age must be a number!)
- Negative Age: Age: -5 (Age must be greater than 0!)

SAMPLE PYTHON CODE:

```
1 import datetime
2 while True:
3     name = input("Enter name: ").strip()
4     age_input = input("Enter age: ").strip()
5     # Validation
6     if not name:
7         print("Name cannot be empty!")
8         continue
9     if not age_input.isdigit() or int(age_input) <= 0:
10        print("Invalid age. Enter a positive number!")
11        continue
12    age = int(age_input)
13    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
14    data = f"{name},{age},{timestamp}\n"
15    # Append to file
16    with open("users.txt", "a") as f:
17        f.write(data)
18    print("Data saved successfully!")
19    choice = input("Add another? (y/n): ").lower()
20    if choice != 'y':
21        break
22    print("Thank you! Data written to users.txt")
```

EXAMPLE OUTPUT (CLI):

```
Enter name: Ali
Enter age: 20
Data saved successfully!
Add another? (y/n): y
Enter name: Siti
Enter age: 19
Data saved successfully!
Add another? (y/n): y
Enter name: John
Enter age: 22
Data saved successfully!
Add another? (y/n): n
Thank you! Data written to users.txt
```

KEY TAKEAWAYS:

- ✓ Use "w" to overwrite, "a" to append.
- ✓ Always validate user input.
- ✓ Add timestamps for better records.
- ✓ Data is stored permanently in files.

Step-by-Step with Code

Step 1 — Write to File (Overwrite Mode)

```
with open("output.txt", "w") as file:  
    file.write("Hello Python\n")
```

- ✓ Creates file and writes content
- ⚠ Overwrites existing content

Step 2 — Append to File

```
with open("output.txt", "a") as file:  
    file.write("New line added\n")
```

- ✓ Adds content without deleting existing data

Step 3 — Save User Input

```
name = input("Enter your name: ")  
  
with open("users.txt", "a") as file:  
    file.write(name + "\n")  
  
print("Data saved")
```

Step 4 — Save Multiple Inputs

```
name = input("Enter name: ")
age = input("Enter age: ")

with open("users.txt", "a") as file:
    file.write(name + "," + age + "\n")
```

Step 5 — Format Output

```
name = input("Enter name: ")
age = input("Enter age: ")

with open("users.txt", "a") as file:
    file.write(f"Name: {name}, Age: {age}\n")
```

Step 6 — Full File Writer Program

```
print("=== User Data Logger ===")

while True:
    name = input("Enter name: ")
    age = input("Enter age: ")
```

```
with open("users.txt", "a") as file:
    file.write(f"{name},{age}\n")

print("Data saved")

choice = input("Add another? (y/n): ").lower()

if choice == "n":
    print("Exiting...")
    break
```

Expected Output (File Content)

```
Ali,20
Siti,19
John,22
```

Optional Exercise

Enhance the program to:

- validate empty input
- prevent invalid age input
- save timestamp with each entry

Answer

```
import datetime

print("=== Advanced User Data Logger ===")

while True:
    name = input("Enter name: ").strip()
    age_input = input("Enter age: ").strip()

    if name == "":
        print("Name cannot be empty")
        continue

    if not age_input.isdigit():
        print("Invalid age input")
        continue

    age = int(age_input)
```

```
timestamp = datetime.datetime.now()

with open("users.txt", "a") as file:
    file.write(f"{name},{age},{timestamp}\n")

print("Data saved")

choice = input("Add another? (y/n): ").lower()

if choice == "n":
    print("Exiting...")
    break
```

Key Takeaways

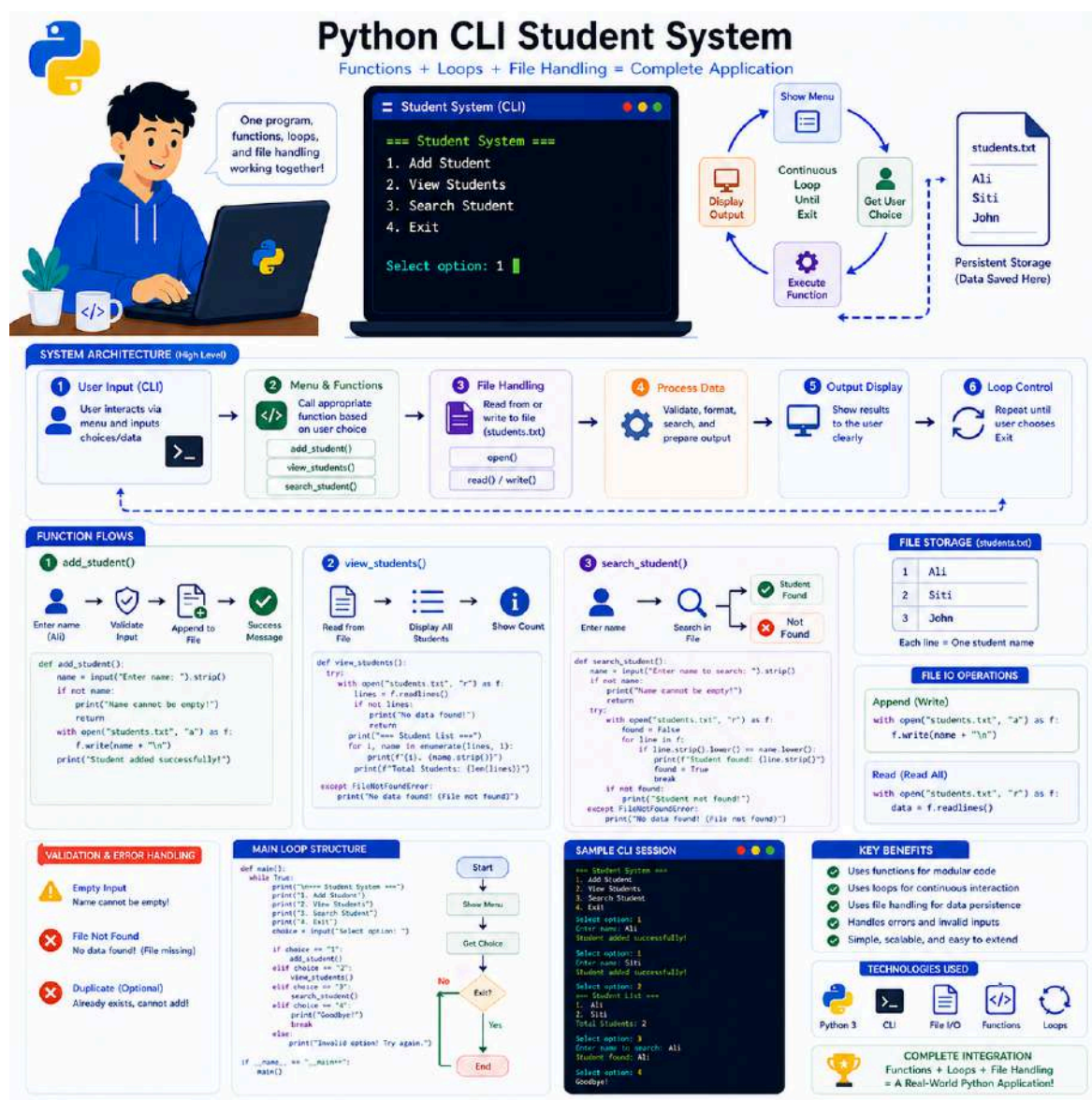
- "w" overwrites files
- "a" appends data
- File writing stores persistent data
- Use formatting for structured storage
- Combine input + file handling for real apps
- Foundation for databases and logging systems

LAB 20 — Pre-Project Integration

Objective

By the end of this lab, you will be able to:

- Combine functions, loops, and file handling
- Build a structured CLI application
- Store and retrieve data from file
- Organize code into reusable components
- Prepare for final project development



Step-by-Step with Code

Step 1 — Create File Path

```
FILE_NAME = "students.txt"
```

Step 2 — Create Function to Add Data

```
def add_student():  
    name = input("Enter student name: ")  
  
    with open(FILE_NAME, "a") as file:  
        file.write(name + "\n")  
  
    print("Student added")
```

Step 3 — Create Function to View Data

```
def view_students():  
    try:  
        with open(FILE_NAME, "r") as file:
```

```
print("\n=== Student List ===")

for line in file:

    print("-", line.strip())

except FileNotFoundError:

    print("No data found")
```

Step 4 — Create Function to Search Data

```
def search_student():

    name = input("Enter name to search: ")

    try:

        with open(FILE_NAME, "r") as file:

            found = False

            for line in file:

                if name.lower() == line.strip().lower():

                    print("Student found")

                    found = True

                    break

            if not found:

                print("Student not found")
```



```
except FileNotFoundError:  
    print("No data file found")
```

Step 5 — Create Menu Function

```
def show_menu():  
    print("\n=== Student System ===")  
    print("1. Add Student")  
    print("2. View Students")  
    print("3. Search Student")  
    print("4. Exit")
```

Step 6 — Main Program (Loop + Functions)

```
while True:  
    show_menu()  
  
    choice = input("Select option: ")  
  
    if choice == "1":  
        add_student()  
    elif choice == "2":
```

```
        view_students()

    elif choice == "3":

        search_student()

    elif choice == "4":

        print("Exiting program...")

        break

    else:

        print("Invalid option")
```

Step 7 — Full Integrated Program

```
FILE_NAME = "students.txt"

def add_student():

    name = input("Enter student name: ").strip()

    if name == "":

        print("Name cannot be empty")

        return

    with open(FILE_NAME, "a") as file:

        file.write(name + "\n")
```

```
print("Student added")
```

```
def view_students():
```

```
    try:
```

```
        with open(FILE_NAME, "r") as file:
```

```
            print("\n=== Student List ===")
```

```
            for line in file:
```

```
                print("-", line.strip())
```

```
    except FileNotFoundError:
```

```
        print("No data found")
```

```
def search_student():
```

```
    name = input("Enter name to search: ").strip()
```

```
    try:
```

```
        with open(FILE_NAME, "r") as file:
```

```
            found = False
```

```
            for line in file:
```

```
                if name.lower() == line.strip().lower():
```

```
        print("Student found")

        found = True

        break

    if not found:

        print("Student not found")

except FileNotFoundError:

    print("No data file found")


def show_menu():

    print("\n=== Student System ===")

    print("1. Add Student")

    print("2. View Students")

    print("3. Search Student")

    print("4. Exit")


while True:

    show_menu()

    choice = input("Select option: ")
```

```
if choice == "1":  
    add_student()  
elif choice == "2":  
    view_students()  
elif choice == "3":  
    search_student()  
elif choice == "4":  
    print("Exiting program...")  
    break  
else:  
    print("Invalid option")
```

Expected Output

```
=== Student System ===  
  
1. Add Student  
2. View Students  
3. Search Student  
4. Exit  
  
Select option: 1  
Enter student name: Ali  
Student added
```

Select option: 2

=== Student List ===

- Ali

Optional Exercise

Enhance the system to:

- delete student
- count total students
- prevent duplicate names

Answer

```
FILE_NAME = "students.txt"
```

```
def add_student():
```

```
    name = input("Enter student name: ").strip()
```

```
    if name == "":
```

```
        print("Name cannot be empty")
```

```
        return
```

```
    # Check duplicates
```

```
    try:
```

```
with open(FILE_NAME, "r") as file:

    for line in file:

        if name.lower() == line.strip().lower():

            print("Student already exists")

            return

except FileNotFoundError:

    pass

with open(FILE_NAME, "a") as file:

    file.write(name + "\n")

print("Student added")

def view_students():

    try:

        with open(FILE_NAME, "r") as file:

            students = file.readlines()

            print("\n=== Student List ===")

            print("Total:", len(students))

            for line in students:
```

```
        print("-", line.strip())
except FileNotFoundError:
    print("No data found")

def delete_student():
    name = input("Enter name to delete: ").strip()

    try:
        with open(FILE_NAME, "r") as file:
            lines = file.readlines()

        with open(FILE_NAME, "w") as file:
            found = False
            for line in lines:
                if name.lower() != line.strip().lower():
                    file.write(line)
            else:
                found = True

        if found:
            print("Student removed")
        else:
```



```
        print("Student not found")

    except FileNotFoundError:
        print("No data file found")

def show_menu():
    print("\n=== Student System ===")
    print("1. Add Student")
    print("2. View Students")
    print("3. Delete Student")
    print("4. Exit")

while True:
    show_menu()

    choice = input("Select option: ")

    if choice == "1":
        add_student()
    elif choice == "2":
        view_students()
```

```
elif choice == "3":  
    delete_student()  
  
elif choice == "4":  
    print("Exiting program...")  
    break  
  
else:  
    print("Invalid option")
```

Key Takeaways

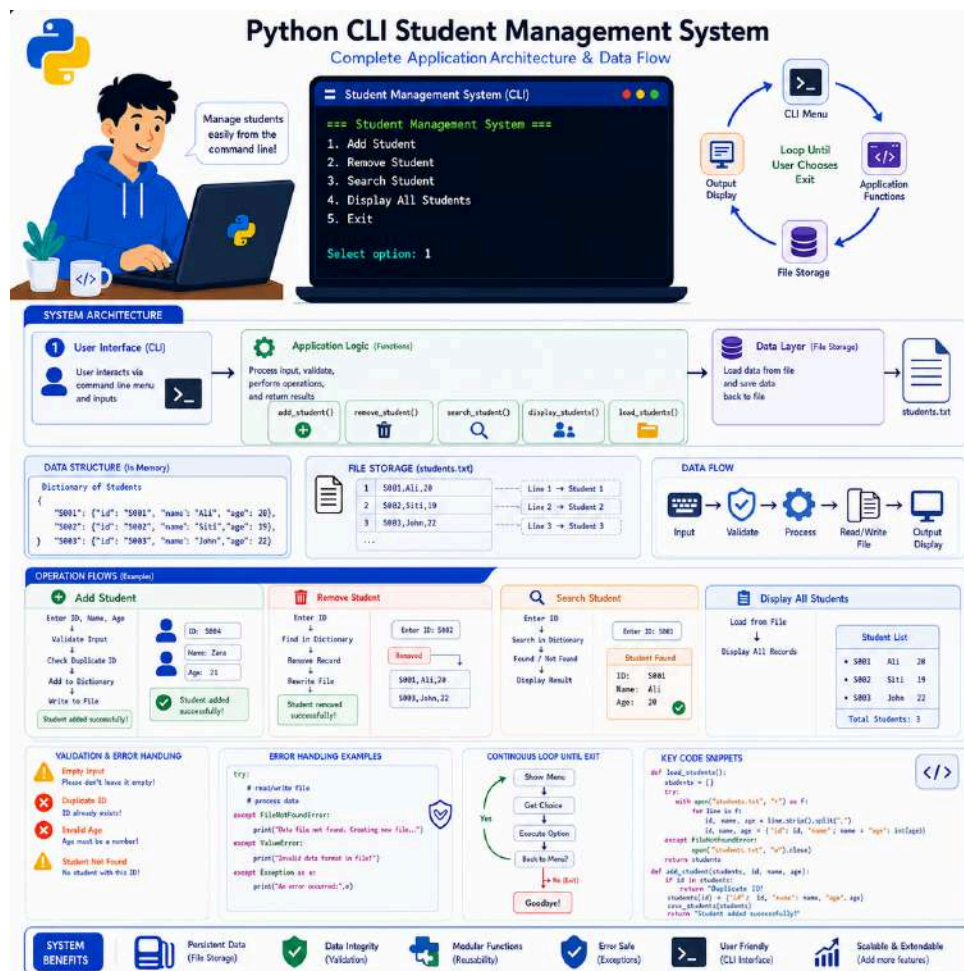
- Functions organize program logic
- Loops enable continuous interaction
- File handling stores persistent data
- Combining concepts builds real applications
- CLI systems simulate backend workflows
- This is a foundation for full project development

FINAL APPLICATION PROJECT — Student Management CLI System

Objective

By the end of this project, you will be able to build a complete Python CLI application that can:

- Add student records
- Remove student records
- Search student records
- Display all students
- Save data into a file
- Handle errors safely
- Use functions, lists, dictionaries, loops, and validation



Step-by-Step with Code

Step 1 — Create Project File

Create a Python file:

```
student_management.py
```

Step 2 — Define File Name

```
FILE_NAME = "students.txt"
```

This file will store student records.

Step 3 — Create Menu Function

```
def show_menu():  
    print("\n=== Student Management System ===")  
    print("1. Add Student")  
    print("2. Remove Student")  
    print("3. Search Student")  
    print("4. Display All Students")  
    print("5. Exit")
```

Step 4 — Create Add Student Function

```

def add_student():

    student_id = input("Enter student ID: ").strip()

    name = input("Enter student name: ").strip()

    age = input("Enter student age: ").strip()


    if student_id == "" or name == "" or age == "":

        print("All fields are required.")

        return


    if not age.isdigit():

        print("Age must be a number.")

        return


    student = {

        "id": student_id,

        "name": name,

        "age": age

    }


    with open(FILE_NAME, "a") as file:

        file.write(f"{student['id']},{student['name']},{student['age']}\n")


    print("Student added successfully.")

```

Step 5 — Create Load Students Function

```
def load_students():  
    students = []  
  
    try:  
        with open(FILE_NAME, "r") as file:  
            for line in file:  
                data = line.strip().split(",")  
  
                if len(data) == 3:  
                    student = {  
                        "id": data[0],  
                        "name": data[1],  
                        "age": data[2]  
                    }  
                    students.append(student)  
  
    except FileNotFoundError:  
        pass  
  
    return students
```

Step 6 — Display All Students

```
def display_students():  
    students = load_students()  
  
    if len(students) == 0:  
        print("No students found.")  
        return  
  
    print("\n=== Student List ===")  
  
    for student in students:  
        print(f"ID: {student['id']} | Name: {student['name']} | Age: {student['age']}")
```

Step 7 — Search Student

```
def search_student():  
    search_id = input("Enter student ID to search: ").strip()  
    students = load_students()  
  
    for student in students:  
        if student["id"] == search_id:
```

```
print("\nStudent Found")

print(f"ID : {student['id']}")

print(f"Name: {student['name']}")

print(f"Age : {student['age']}")

return

print("Student not found.")
```

Step 8 — Remove Student

```
def remove_student():

    remove_id = input("Enter student ID to remove: ").strip()

    students = load_students()

    updated_students = []

    found = False

    for student in students:

        if student["id"] == remove_id:

            found = True

        else:

            updated_students.append(student)

    if not found:
```



```
print("Student not found.")

return

with open(FILE_NAME, "w") as file:

    for student in updated_students:

        file.write(f"{student['id']},{student['name']},{student['age']}\n")

print("Student removed successfully.")
```

Step 9 — Create Main Program Loop

```
while True:

    show_menu()

    choice = input("Select option: ").strip()

    if choice == "1":

        add_student()

    elif choice == "2":

        remove_student()

    elif choice == "3":

        search_student()

    elif choice == "4":
```

```
        display_students()

    elif choice == "5":

        print("Exiting system...")

        break

    else:

        print("Invalid option. Please try again.")
```

Full Final Code

```
FILE_NAME = "students.txt"

def show_menu():

    print("\n=== Student Management System ===")

    print("1. Add Student")

    print("2. Remove Student")

    print("3. Search Student")

    print("4. Display All Students")

    print("5. Exit")

def load_students():
```

```
students = []

try:
    with open(FILE_NAME, "r") as file:
        for line in file:
            data = line.strip().split(",")

            if len(data) == 3:
                student = {
                    "id": data[0],
                    "name": data[1],
                    "age": data[2]
                }
                students.append(student)

except FileNotFoundError:
    pass

return students

def add_student():
    student_id = input("Enter student ID: ").strip()
```

```
name = input("Enter student name: ").strip()
age = input("Enter student age: ").strip()

if student_id == "" or name == "" or age == "":
    print("All fields are required.")
    return

if not age.isdigit():
    print("Age must be a number.")
    return

students = load_students()

for student in students:
    if student["id"] == student_id:
        print("Student ID already exists.")
        return

student = {
    "id": student_id,
    "name": name.title(),
    "age": age
}
```

```
try:
```

```
    with open(FILE_NAME, "a") as file:
```

```
        file.write(f"{student['id']},{student['name']},{student['age']}\n")
```

```
    print("Student added successfully.")
```

```
except Exception as error:
```

```
    print("Error saving student:", error)
```

```
def display_students():
```

```
    students = load_students()
```

```
    if len(students) == 0:
```

```
        print("No students found.")
```

```
        return
```

```
    print("\n=== Student List ===")
```

```
    for student in students:
```

```
        print(f"ID: {student['id']} | Name: {student['name']} | Age: {student['age']}")
```

```
def search_student():  
    search_id = input("Enter student ID to search: ").strip()  
    students = load_students()  
  
    for student in students:  
        if student["id"] == search_id:  
            print("\nStudent Found")  
            print(f"ID : {student['id']}")  
            print(f"Name: {student['name']}")  
            print(f"Age : {student['age']}")  
            return  
  
    print("Student not found.")  
  
def remove_student():  
    remove_id = input("Enter student ID to remove: ").strip()  
    students = load_students()  
  
    updated_students = []  
    found = False
```

```
for student in students:

    if student["id"] == remove_id:

        found = True

    else:

        updated_students.append(student)

if not found:

    print("Student not found.")

    return

try:

    with open(FILE_NAME, "w") as file:

        for student in updated_students:

            file.write(f"{student['id']},{student['name']},{student['age']}\n")

    print("Student removed successfully.")

except Exception as error:

    print("Error removing student:", error)

while True:

    show_menu()
```

```
choice = input("Select option: ").strip()

if choice == "1":
    add_student()
elif choice == "2":
    remove_student()
elif choice == "3":
    search_student()
elif choice == "4":
    display_students()
elif choice == "5":
    print("Exiting system...")
    break
else:
    print("Invalid option. Please try again.")
```

Expected Output

```
=== Student Management System ===
1. Add Student
2. Remove Student
3. Search Student
```


4. Display All Students

5. Exit

Select option: 1

Enter student ID: S001

Enter student name: ali

Enter student age: 20

Student added successfully.

Optional Exercise

Enhance the system to include:

- Update student details
- Validate age range between 5 and 100
- Save data in CSV format with header
- Add confirmation before deleting student

Answer — Age Range Validation Example

```
if not age.isdigit():  
    print("Age must be a number.")  
    return  
  
age_number = int(age)
```

```
if age_number < 5 or age_number > 100:  
    print("Age must be between 5 and 100.")  
  
    return
```

Key Takeaways

- Functions keep the program organized
- Lists store multiple student records
- Dictionaries represent each student
- File handling stores data permanently
- Exception handling prevents crashes
- Validation improves system reliability